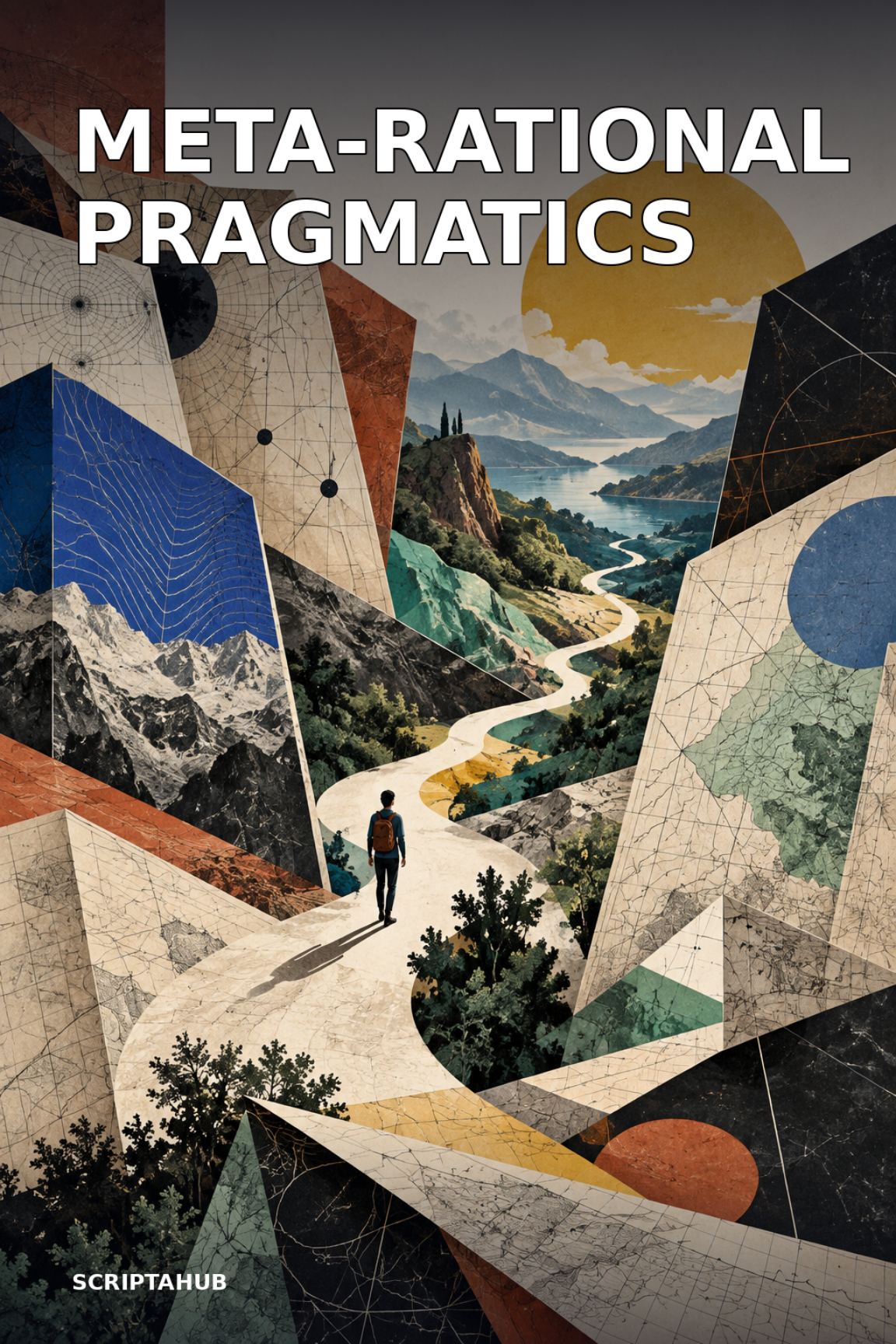
# META-RATIONAL PRAGMATICS



SCRIPTAHUB

RESEARCH PROGRAM

# META-RATIONAL PRAGMATICS

## Understanding as Learning Executable Distinctions

Foundations, Architecture and Applications

**Sînică Alboaie**

Research Conception and Intellectual Direction
AI-Assisted Development

COURSE • OPERATIONAL MODEL • PROPOSED EXPERIMENTS

Study Edition | September 2026

# Contents

Preface. A Hypothesis That Must Be Able to Lose ..... 4

**I / The Question and the Language**

1. Understanding as an Engineering Problem ..... 6

2. A Minimal Mathematical Vocabulary ..... 9

3. Programs, Semantics, Interpretation ..... 12

4. Probability, Decision, Truth ..... 15

**II / Foundations for Distinctions**

5. Representations and Equivalences ..... 18

6. Learning States ..... 21

7. Prediction and Intervention ..... 24

8. Abstract Interpretations ..... 27

**III / A Learning Mechanism**

9. Diagnosing Errors ..... 30

10. Interpreter Induction ..... 33

11. The Rationality of Computation ..... 36

12. Memory and Transfer ..... 39

# Contents · Continued

**IV / Forms of Knowledge**

13. Pragmatic Distances ..... 42

14. Cones and Coordinates ..... 45

15. An Atlas of Interpreters ..... 48

**V / From Model to System**

16. The Architecture of an MRP Technology ..... 51

17. Execution, Evidence and Revision ..... 54

18. MRP-VM and SOP Lang ..... 57

**VI / Applications and Validation**

19. Software and Protocol Verification ..... 60

20. Documents, Synthesis and Procedures ..... 63

21. Research and Discovery ..... 66

22. Education and Interactive Agents ..... 69

23. The Experimental Program ..... 72

24. Limits, Objections and Contributions ..... 76

**Appendices / Tools for Study and Verification**

A. Elementary Mathematical Results ..... 80

B. Exercises with Explained Solutions ..... 84

C. Operational Glossary ..... 87

D. Annotated Bibliography ..... 90

# A Hypothesis That Must Be Able to Lose

A technology of understanding does not need a definitive definition of the mind to begin. It needs situations in which an agent confuses things it should distinguish, operations through which it can correct that confusion, and independent criteria for checking whether it has progressed. This book develops Meta-Rational Pragmatics, or MRP, starting from that requirement.

The central intuition is that an agent's understanding does not reside exclusively in a repository of representations. It also depends on the procedures through which the agent constructs representations, uses them, compares them with experience and replaces them. Some procedures interpret a text; others track a sequence of events, estimate an outcome, verify a proof or produce a new procedure. In this sense, the agent accumulates not only answers but ways of making certain differences visible.

The book turns this intuition into an engineering proposal: an adaptive family of interpreters controlled through tests, counterexamples, costs and evidence. The expression “executable distinction” names a difference a program can compute and whose relevance can be checked. For example, the order in which permission is granted and revoked is an executable distinction. The convincing tone of an explanation is not, by itself, evidence that the agent preserved it.

The work is organized as a progressive course. It introduces mathematical objects before using them; explains equivalence relations before the learned automaton; separates probability from utility before metareasoning; and introduces distances and convexity before cones. Readers need not know abstract interpretation, predictive representations or program synthesis. Elementary familiarity with programs and tables is useful, but the examples do not require reading any particular programming language.

MRP is presented neither as a validated theory of human cognition nor as a product already built. Established elements are attributed to the literature. Elementary mathematical results are proved within the limits of their assumptions. Architectural components are proposals. Numerical

examples are teaching calculations, not prototype measurements. Applications are analyzed as testing opportunities, not fulfilled commercial promises.

| Status | What It Means in This Book | What It Does Not Authorize |
|---|---|---|
| Established Foundation | An idea or method documented in the cited literature. | Automatic transfer of all its guarantees to MRP. |
| Derived Result | A conclusion proved under explicit assumptions. | Generalization beyond those assumptions. |
| Proposed Mechanism | A design rule precise enough to implement. | The claim that the implementation exists or is competitive. |
| Empirical Hypothesis | A prediction evaluable through comparison and experiment. | Treating a coherent intuition as an observed result. |

Citations use identifiers of the form [Cousot1977]. The bibliography supplies the works and their roles in the argument. It is selective: it supports the mechanisms discussed without claiming an exhaustive inventory of all research on agents or understanding. The name MRP designates the program formulated here; using it does not claim priority over metareasoning, pragmatism or program learning.

The research conception belongs to Sînică Alboaie. Conceptual development, exposition and bibliographic verification were assisted by AI. The text has not undergone independent scientific review. Readers are invited to retain mechanisms that withstand criticism and reject formulations that produce no measurable differences.

CHAPTER 01

# Understanding as an Engineering Problem

## From Impression to Capability

Suppose an assistant receives two records: “access granted, then revoked” and “access revoked, then granted.” It identifies the same people, resources and two operations in both records. Yet if the system's rule is that the last operation prevails, the two situations do not permit the same decision. A rich vocabulary about access does not compensate for losing the order.

This example establishes a narrower question than “does the agent really understand?” The question is whether the agent has the representation and procedure needed to answer a family of questions correctly. We will call this property operational understanding. The term does not resolve disputes about consciousness, intentionality or subjective experience; it selects a dimension that can be investigated through systems that are built and compared.

The difference between the two questions matters methodologically. For the first, identical behaviors can receive different philosophical interpretations. For the second, we can construct examples with known answers, modify the available information and observe which component enables progress. This narrowing does not make the problem trivial. Discovering what must be represented without the designer supplying every distinction is precisely the difficult problem.

## Three Different Theses

The first thesis says that any intelligent behavior can be described through a family of interpreters. This is plausible as an organizing perspective, but too permissive to function alone as an empirical hypothesis. Any sufficiently complex program can be retrospectively divided into modules and described this way.

The second thesis says that certain errors arise because the available representation identifies situations requiring different outcomes. This thesis is precise and admits local proofs. If two inputs are transformed into exactly the same value and a subsequent mechanism sees only that value, the mechanism cannot respond differently based on information already lost.

The third thesis is MRP's wager: in certain problem families, an agent that detects such collisions and learns reusable interpreters will achieve a better trade-off between error, cost and adaptability than comparable alternatives. This does not follow logically from the first two theses. It requires a concrete architecture and experiments that include every cost.

These three levels must not be mixed. A proof about lost information does not establish a product's superiority. A useful prototype in a microworld does not establish a universal theory of the mind. Conversely, a debatable philosophical claim does not prevent testing the engineering mechanism.

## Representation and Interpretation

A representation is a structure the agent can store or manipulate: a number, table, vector, graph, expression or program. An interpreter is a procedure that assigns operational consequences to such structures: answers, transformations, predictions or actions. The two are connected but not identical.

An event table does not produce the current state by itself. A rule for ordering and updating is required. The same rule applied to a table lacking timestamps or order indicators may become unusable. The capability belongs to the representation–operation pair, not either half in isolation.

The symbol grounding problem, formulated by Harnad, asks how symbols can be connected to something beyond other symbols [Harnad1990]. MRP does not claim to settle this philosophical problem. It addresses it practically through verifiable links between representations, sources, sensors and the effects of actions. An identifier in a report becomes operational when it can be traced to the observation or object it refers to.

This grounding can be indirect. An agent analyzing documents need not have its own sensors for every claim. But it must distinguish “I observed,” “the source states,” “it follows under the model's assumptions” and “I propose as a possibility.” These distinctions belong to the architecture, rather than being mere polite phrases.

## Pragmatic Does Not Mean Opportunistic

In this book, pragmatics concerns using an interpretation within a context, for a purpose and under constraints. It does not mean a proposition becomes true if it is convenient. A system that learns to obtain user approval through false answers may succeed under a poorly defined reward and fail epistemically.

For this reason, the design must separate three evaluations. Correctness checks whether the result follows the assumed rules. Empirical adequacy checks whether predictions match observations. Utility checks whether the decision serves the task's legitimate purpose. A single score may be useful in a well-bounded experiment, but must not hide the differences between these evaluations.

“Meta-rational” names the choice and development of a way of reasoning. An agent may decide that a numerical comparison is sufficient, that time must be tracked or that an observation is missing. It is not a magical logic above all other logics. It is the management of finite cognitive and computational resources.

## A Working Definition

MRP will designate a mechanism through which an agent constructs, selects, composes and revises interpreters using verifiable consequences and declared costs. MRP progress occurs when this reorganization produces competence on new cases, rather than merely correcting the case that triggered it.

The definition leaves implementations open. Interpreters may be symbolic, numerical, neural or hybrid. Some may be supplied by the designer; others may be learned. To avoid confusion, every experiment will separately report what was provided, what was induced and what was merely selected. This accounting is the first condition of a defensible result.

CHAPTER 02

# A Minimal Mathematical Vocabulary

## Sets, Functions, Relations

A set is a collection of objects treated as distinct elements. In our example, the set of possible events may contain granting and revoking access. A sequence differs from a set: it preserves order and usually repetitions. The sequence “grant, revoke” is not the sequence “revoke, grant,” although the sets of their elements are identical.

A function associates a single result with every admissible input. The notation R(h) means the result of representation function R applied to history h. If R preserves only the set of events, the two histories have the same image. A partial function is not defined for every input; for example, “the last event” does not exist for an empty sequence unless an additional convention is introduced.

A relation need not provide a unique result. The relation “depends on” can connect a result to multiple sources. The relation “is compatible with” can connect an observation to several models. Distinguishing functions from relations allows us to preserve ambiguities without prematurely turning them into an arbitrary choice.

Composition executes one transformation after another. If E extracts the events and R reduces them to a state, R(E(text)) denotes the entire path. Composition is valid only if the first operation's result is an admissible input to the second. This mathematically simple condition will become an important contract between interpreters.

## Equivalence and Partition

An equivalence relation is reflexive, symmetric and transitive. Reflexivity requires every object to be equivalent to itself. Symmetry requires equivalence to work in both directions. Transitivity requires that if the first object is equivalent to the second and the second to the third, the first is equivalent to the third.

Such a relation divides the domain into disjoint classes called equivalence classes. The collection of classes is a partition. Grouping histories by the result R(h) automatically yields a partition: the same R value means the same class. A representation can therefore be understood as a way of specifying which differences we ignore.

One partition refines another when it separates some classes without merging elements already separated. For example, classifying documents by type and year refines classification by type alone. It can preserve old information and add new information. This does not mean it will automatically produce a better algorithm: it may increase cost and require more estimation data.

We will sometimes use the term “quotient space.” It denotes the set of equivalence classes, not a special geometric space. We replace each concrete history with the class of all histories considered equivalent. Geometry, if any, must subsequently be added through a distance or another structure.

## Orders and Graphs

A partial order allows some pairs to be compared without requiring every pair to be comparable. An interpreter preserving event order and another preserving documentary sources may be incomparable: each conserves information the other ignores. There need not be a single sequence of representations from “bad” to “good.”

A graph consists of nodes and links. In MRP, nodes may be operations or results; links may represent dependencies. If a computation must first receive two results, the graph specifies that condition. A directed graph without cycles allows operations to be ordered so that each follows its dependencies. This is called a topological ordering.

A cycle is not necessarily an error. It may describe repeating an update until stabilization. But a rule must explain what repeats, which state changes and when the process stops. A circular drawing is not yet an executable algorithm.

## Predicate and Invariant

A predicate is a property that can be true or false of an object within the chosen model. “Every event has an order index” is a predicate on a table. An invariant is a property that must remain true throughout an execution or a class of transformations.

For an MRP library, one possible invariant is that every accepted result preserves the identifiers of the sources used. This does not mean the sources are true; it means the result's origin is not lost. Another invariant may require an operation declared read-only not to modify external state.

A proof starts from assumptions and uses explicit rules to obtain a conclusion. A test executes one or more cases. A proof may cover every case in the model; a test does not acquire that power merely by being automated. In turn, a proof does not by itself check whether the model correctly describes the real world.

## Cost and Size

A problem's size may be the number of events, the number of candidate interpreters or the maximum length of programs being searched. A linear procedure visits a number of elements proportional to the size. An exponential one may double the search space with every unit increase. These differences explain why a logically correct idea can be impractical.

Computability and complexity distinguish the existence of an algorithm from the resources needed to execute it. Introductory treatments of theoretical computer science develop this distinction systematically [Barak]. For MRP, it translates into a requirement: every induction mechanism must have search, time and memory limits, even if its conceptual description is very general.

| **Mathematical Object** | **MRP Example** | **The Practical Question** |
|---|---|---|
| Function | Reducing history to the current state. | Which inputs does it accept, and what does it lose? |
| Partition | Histories with the same representation. | Which situations are confused? |
| Predicate | The existence of a complete ordering. | Can it be checked before execution? |
| Graph | Dependencies between operations. | What must be computed or invalidated? |
| Partial Order | Comparing preserved information. | Which representation can reproduce another? |

This vocabulary is not decoration. It will let us turn expressions such as “the agent sees the problem differently” into claims about functions, classes, programs and observable results.

CHAPTER 03

# Programs, Semantics, Interpretation

## What Semantics Establishes

Syntax determines which expressions are well formed. Semantics determines how they are interpreted. In a programming language, it may describe state transitions, an execution's result or the properties commands preserve. Semantics must therefore not be reduced to a static dictionary of meanings. Abstract interpretation explicitly offers multiple semantics and relations between them [Cousot1977].

Consider a command that adds one to a number. Its semantics must specify whether the number is an unbounded mathematical integer or an integer represented by a fixed number of bits. At the maximum representable value, the answers may differ. Simply saying “it understands addition” conceals an essential modeling choice.

Operational semantics describes execution steps. Denotational semantics associates a program with a mathematical object describing its behavior. Axiomatic semantics expresses relationships between input conditions and output properties. These perspectives can complement one another. MRP does not replace them with a broader term; it treats them as tools available to the agent.

## What Pragmatics Adds

A semantic rule does not always decide by itself whether it is worth applying, which information is missing or which goal must be satisfied. Here we introduce the pragmatic level: choosing the interpretation regime, relevant context and stopping criterion. Once chosen, the interpretation's rules must remain stable enough for verification.

We can call an interpretation whose domain and obligations are fixed for an execution “stabilized semantics.” This is a convention internal to the book, not a universally accepted classification in the philosophy of language. It enables a useful principle: the agent may change interpretation between revision episodes, but cannot tacitly change the meaning of rules in the middle of a proof.

For example, a system may switch from mathematical integers to 32-bit arithmetic. This is not merely an optimization if it changes results. It changes the model; old conclusions must be reexamined wherever the difference matters.

## The Interpreter as an Object

In the proposed model, an interpreter receives input, context, goal and internal state. It produces a result, a new internal state and evidence. Here “evidence” will denote verifiable support: sources, tests, computational traces or proofs, rather than merely the agent's expressed conviction.

Interpret(input, context, goal, state) = (result, newState, evidence).

This expression is a conceptual interface, not a complete algorithm. Its fields must be restricted for each concrete interpreter. An order analyzer may require a table with unique indices. A verifier may require a formula in a precise logic. A generator may accept text, but its output will initially have proposal status.

Context must not become a place to hide unlimited information. If the selector can consult the full text while the baseline predictor receives only a summary, the comparison measures more than interpreter quality. Every experiment will declare exactly what each component can read.

## Contracts and Composition

A contract specifies preconditions, promised results and permitted effects. Hoare-style logic expresses the relationship between a precondition, a command and a postcondition. Partial correctness says that if execution terminates and the precondition held, the postcondition holds; termination requires a separate obligation [SoftwareFoundations].

Consider an operation selecting a resource's last event. Its contract may require a nonempty history, a total ordering and unambiguous resource identification. The postcondition says the returned event belongs to the resource and is followed by no other event for that resource. This postcondition can be checked independently of how the operation was implemented.

Composing two interpreters requires more than matching formats. If the first returns “the last known event,” the second cannot assume “the last event that occurred in the world” without assuming the history is complete. This is an example of semantic incompatibility in an apparently technically correct interface.

Probabilistic interpreters require more cautious contracts. “It frequently produced correct answers” is not a universal logical postcondition. We can describe the tested domain, evaluation distribution and performance uncertainty. Composing probabilistic components does not turn those results into certainty.

## Learning as an Interpreter

A learning procedure interprets data through a family of models and an adjustment criterion. It produces parameters, rules or programs. It can therefore be included in the same interface schema. Yet this does not make every operation identical: changing a temporary result differs from changing the library on which future tasks will depend.

We will separate working state from consolidated state. A new hypothesis can exist in an experimental branch without becoming an accepted rule. Promoting it requires tests and an explicit decision. This separation lets the agent explore without treating exploration as truth already acquired.

## What Remains Stable

An architecture capable of modifying itself needs a core stable enough to determine which modifications have occurred. The core may be small: version management, execution rules, evidence types and cost accounting. It need not contain every domain concept in advance.

MRP becomes interesting if its core enables useful interpreters to develop without rewriting the entire system. Manually supplied components are legitimate, but must be declared separately from learned ones.

CHAPTER 04

# Probability, Decision, Truth

## Uncertainty Is Not Contradiction

An agent may not know whether access is available even when the real system has a precise state. An event may be missing, a source unreliable or the policy changed. Not knowing the state and a contradiction between two statements are different situations. The architecture must represent them separately.

A probability is a value between zero and one assigned to an outcome within a model. For mutually exclusive outcomes covering every possibility considered, probabilities sum to one. A distribution is the collection of these values. It describes the agent's uncertainty or a mechanism's regularity, depending on the model used.

The mean of a random variable is not the outcome that will necessarily occur. In a finite domain, it is the sum of possible outcomes multiplied by their probabilities. An unlikely outcome may still occur; a well-calibrated agent is not one whose individual predictions always come true.

## Updating a Belief

Consider a fictional example: before a test, the agent assigns a probability of 0.2 to a fault being present. A signal occurs in 90% of faulty cases and 10% of cases without a fault. After the signal appears, the updated probability is 0.18 divided by 0.26, approximately 0.692.

The calculation compares two ways the signal could have appeared: 0.2 × 0.9 of cases arise from faults, while 0.8 × 0.1 do not. The new result is a conditional probability, not a proof. It depends on the correctness of the initial rates and their applicability to the current situation.

An MRP interpreter can perform this update. Another can challenge the model supplying the rates. Changing numbers within a model and changing the model are different operations; the second may require additional experience, rather than merely recalculating the first.

## Loss and Choice

A loss function assigns a cost to a decision in a situation. If incorrectly granting access causes a loss of 100 and incorrectly denying it a loss of 1, the rational threshold is not a probability of 0.5. Assuming zero cost for correct decisions and probability p that access is legitimate, granting access has expected loss $100 \times (1 - p)$, while denying it has loss p.

Granting access is preferable only when p exceeds 100/101, approximately 0.9901. This is a calculation about a fictional cost matrix, not an access-policy recommendation. It shows why the same uncertainty can lead to different decisions under different objectives.

Preferences do not, however, change the distribution of truth. The threshold may legitimately be changed; the probability must not be falsified to justify the desired decision. A transparent system can state: “I consider the event likely, but the cost of an error requires further checking.”

## Evaluating Probabilities

Proper scoring rules are designed so that honest probabilistic prediction optimizes the expected score. Gneiting and Raftery analyze this class of methods and the distinction between calibration and the informative sharpness of predictions [Gneiting2007]. In an MRP experiment, these methods can prevent evaluation from being reduced to merely choosing the majority label.

For a binary outcome y equal to zero or one, a common form of Brier loss is $(p - y)^2$. If the true probability is q, expected loss can be rewritten as $(p - q)^2 + q \times (1 - q)$. The second term does not depend on prediction p. The first is minimized when $p = q$. This derivation explains why the score favors correct probabilities, not merely correct labels.

The score cannot make data representative by itself. An agent evaluated only on easy situations can score excellently and perform poorly precisely in important cases. The test distribution is part of the experiment's specification.

## Abstention and Information

Abstention is an action with its own cost: delay, human intervention or a lost opportunity. It must be treated neither as absolute failure nor as a free solution. A useful technology will report the response rate, error among supplied answers and the cost of referred cases together.

More precise knowledge has value when it can change the decision in a sufficiently important way. A new observation that changes no relevant behavior may have scientific value but little operational value in the current episode. In a research program, learning's long-term value must be accounted for separately from immediate success.

For MRP, probability describes what the agent estimates, loss expresses what matters for the task, and verification establishes what support exists. These are three distinct objects. Keeping them separate enables metareasoning that saves computation without turning economy into permission to invent facts.

CHAPTER 05

# Representations and Equivalences

## What the Agent May Forget

Every finite representation selects. It preserves some differences and ignores others. The problem is not eliminating every loss, but identifying losses incompatible with the task. A summary omitting the cover's color may be perfectly adequate for analyzing arguments. The same summary is insufficient for identifying the edition by its appearance.

We fix a family T of tests. A test may be a question with a verifiable answer or a procedure for interacting with the environment. Initially, we consider deterministic tests. Two histories are equivalent relative to T if every test in T produces the same answer for both histories.

h1 is T-equivalent to h2 exactly when Test(h1) = Test(h2) for every Test in T.

The relation is reflexive, symmetric and transitive because equality of answers has these properties. It produces a partition of histories. This partition specifies which information can be ignored without changing answers to the declared tests. It does not specify what can be ignored for every imaginable future question.

## The Sufficient Representation

A representation R is sufficient for T if every test can be solved using only R(h). In a finite domain without computational restrictions, this is equivalent to requiring that R not merge histories with different answers to a relevant test.

The proof is direct. If two histories receive the same representation, any function seeing only the representation will produce the same result for them. It therefore cannot reproduce two different answers. Conversely, if all answers within each R class are identical, we can assign that answer to the class. This constructs a function on representations that reproduces the test.

The result concerns information. The constructed function may require a very large table or be difficult to compute. Information sufficiency is not the same as efficient access to it. A raw archive may contain everything needed, while a specialized procedure makes the information usable within an acceptable time.

The partition induced by T is the coarsest partition sufficient for these tests. Any other sufficient representation must preserve at least its distinctions. This formulation does not imply unique syntax: the same classes can be encoded through numbers, labels, programs or other objects.

## A Provable Limit

We return to granting and revoking access. Suppose every history is fully available before compression, the last operation determines the state, and the representation preserves only the set of operations. The two orderings receive the same representation. If they occur with equal probabilities, no predictor seeing only that representation can exceed an expected accuracy of 50% on this pair.

A deterministic predictor must choose an answer and is wrong on one case. A randomized predictor saying “yes” with probability p has accuracy 0.5 × p + 0.5 × (1 − p), still 0.5. Randomization does not reconstruct the lost order.

The claim ceases to apply if the predictor sees another information source: memory of the episode, a case identifier correlated with the answer or the raw history. An experiment about information loss must therefore control every input channel, not just the vector the author calls the “representation.”

For a finite labeled dataset, the best classification based exclusively on R chooses the majority label in each R class. The minimum number of errors is the total number of examples minus the sum of the majority frequencies across classes. This bound can be calculated exactly, providing a diagnostic tool before training a more complex predictor.

## Refinement as Learning

If a new function D separates the confused cases, we can define Rnew(h) = (R(h), D(h)). The new representation preserves the old information and adds the distinction. D may be a computable question: the last operation, the number of cancellations, the existence of an exception or the author of a statement.

This is what the expression “a dimension is a program” means. The dimension need not be an uninterpreted real number. It may be a discrete result or a relational structure. The agent learns an operation that changes what it can observe in its own data.

Refinement may require no new external observations, but it is not computationally free. Computing D may require rereading a long archive. If the archive was discarded, D may become impossible to apply retrospectively. Memory design determines which kinds of later learning remain possible.

## What Compression Does Not Guarantee

A representation sufficient for today's questions may be insufficient for tomorrow's. We can manage this limit by preserving sources, using multiple representations and allowing requests for new observations. We cannot eliminate it with a universal assertion that “the agent understood the essence.” Operational essence must be related to a family of uses.

Information theory distinguishes measuring and encoding information from semantically interpreting messages; this boundary is explicit in Shannon's work [Shannon1948]. MRP adds a design question: which information must be conserved for relevant tests and decisions? A highly compressed file may remain difficult to use, while a redundant representation may enable rapid answers.

The final criterion is therefore not “how small is the memory?” but “which competence is preserved, at what construction and usage cost, for which task distribution?” This question will govern both interpreter learning and evaluation.

CHAPTER 06

# Learning States

## A Small Protocol

Consider a fictional protocol with three states: inactive, pending and active. A request moves the system from inactive to pending. A confirmation activates only an existing request. A cancellation returns it to inactive. To avoid ambiguity, a repeated request does not deactivate access that is already active, and confirming active access leaves it active.

The agent does not see the state name. It sees operations and can ask whether access is active. Two histories, the empty history and a history containing a request, produce the same negative answer to the current question. Yet after confirmation, their answers differ. State cannot be identified by the present result alone; responses to continuations matter.

| Representative History | Access Now | Access After Confirmation | State Required by the Model |
|---|---|---|---|
| No Event | No | No | Inactive |
| Request | No | Yes | Pending |
| Request, Confirmation | Yes | Yes | Active |

This is an example of a new test producing a new distinction. The agent does not arbitrarily invent “another state”; it discovers that two situations treated identically behave differently under an executable continuation.

## Automata and Minimal Memory

A finite automaton has finitely many states and a transition rule for every admissible input. Its state summarizes what must be retained about the past for the model's future behavior. An automaton is not a necessary explanation of the brain, but a mathematical object suited to protocols and discrete processes.

For regular languages, the Myhill–Nerode principle identifies histories that no continuation can distinguish with respect to acceptance. In active automata learning, this perspective leads to constructing states from an-

swers to queries. Dana Angluin's L* algorithm is a fundamental reference, and generalizations to reactive systems and decision processes show its relevance beyond simple string recognition [Angluin1987; Tappler2019].

For MRP, the lesson is not that every concept must be reduced to automata. It is that some of the agent's internal entities can be constructed from tests' inability to distinguish histories. A state becomes a hypothesis about future behavior, rather than a manually assigned semantic label.

## Why Closure Matters

If two histories are considered equivalent, applying the same operation must lead them to states that remain equivalent. Otherwise, the transition from their shared class is not well defined. For string continuations, this property is called right congruence.

Equivalence for all relevant continuations has this property: a continuation distinguishing the successors could be preceded by the common operation and distinguish the initial histories. But equivalence for an arbitrarily chosen finite list of tests need not have it. In our protocol, the “access now” test merges inactive and pending states, although confirmation separates them.

A test basis must therefore be evaluated for updating, not merely for the immediate answer. A representation answering correctly today but incapable of correct updating after the next event is not a sufficient state of the dynamic system.

## A Learning Loop

The agent can maintain a table with representative histories as rows and test continuations as columns. Identical rows suggest the same state. When a new continuation produces different answers, the rows are separated. When an operation leads to a row type not yet represented, a new representative must be introduced.

This is a construction mechanism, not merely post-hoc classification. After every extension, the agent builds a candidate model and confronts it with the system. A counterexample is a sequence on which the candidate's behavior differs from the target's. The sequence can be reduced to locate the missing test.

In an ideal setting, an equivalence oracle states whether the candidate model is correct and supplies a counterexample when it is not. In a real system, such an oracle is usually replaced by testing. Tappler and col-

leagues explicitly distinguish a perfect-information setting from learning through sampled traces [Tappler2019]. Failure to find a counterexample must not be reinterpreted as proof of equivalence.

## Resetting and Experimental Access

Comparing two continuations from the same situation may require resetting the system or reproducing a history. This may be simple in a simulator. In an organizational workflow, it may be expensive or impossible. The ability to experiment must not be assumed free.

A learning environment must declare which actions are available, which observations are obtained, whether resetting is possible and whether execution irreversibly changes the world. These conditions determine which algorithms can be applied and which claims can be validated.

## What MRP Gains

A learned automaton can become a reusable interpreter of history. It can process new events incrementally without continually rereading the entire past. But this efficiency assumes that the learned state preserves the necessary differences within the evaluated domain.

The experiment must distinguish inducing a new model from selecting a supplied one. Only the former tests the construction of distinctions; both may have engineering value.

CHAPTER 07

# Prediction and Intervention

## Beyond Visible States

In a probabilistic system, two histories may produce different outcomes when the same test is repeated without necessarily representing different states. The difference may arise from randomness. The equivalence criterion must compare distributions, not a single observation.

We will write Distribution(test, h) for the distribution of test outcomes after history h. Two histories are predictively equivalent relative to T if these distributions are equal for every test in T. Exact equality is a theoretical object; experimentally, we estimate distributions from samples and report uncertainty.

An interactive test includes how actions are chosen. If the first observation determines the next action, the test is an experimental policy, not merely a fixed list. Two systems can be compared only if we use the same test specification and relevant starting conditions.

## Predictive Representations

Predictive state representations, or PSRs, describe state through predictions about the outcomes of tests executable on the system. Singh, James and Rudary develop this idea using a system-dynamics matrix whose rows correspond to histories and columns to tests [Singh2004].

An example makes the intuition clear. Instead of storing a hidden label such as “latent mode 7,” we can store the probability that a simple check succeeds, the probability that two successive operations succeed and the probability that a reset restores functioning. These coordinates are meaningful because they refer to observations that can be collected.

Not every list of predictions is a sufficient PSR. There must be a rule for calculating relevant predictions from it and updating it after actions and observations. Choosing a few intuitive questions does not ensure this property.

## Rank Without Mystery

A matrix is a numerical table. Its rank measures how many independent columns or rows are needed to describe the table's linear information. If some predictions can be obtained as linear combinations of others, they need not all be stored as independent dimensions.

This offers possible compression for suitably structured systems. However, estimating a low rank from finite data does not prove that the real system has exactly that rank. Noise may make columns appear independent, while insufficient tests can hide differences. A useful predictive representation requires both a structural hypothesis and a validation procedure.

The coefficients of a linear combination must not automatically be interpreted as probabilities. They may be negative, although the final prediction must remain a valid probability. Moreover, updating after observing an event usually involves normalization. Linear compression of predictions does not mean the entire agent becomes a simple average of models.

## Observing Is Not Intervening

Consider two models with binary variables. In the first, a hidden variable Z determines both X and Y. In the second, Z determines X and X determines Y. Suppose Z is zero or one with equal probabilities and every link copies the value. In observational data from both models, we always see X = Y.

If we intervene and fix X at zero, the difference emerges. In the first model, Y continues to copy Z and is one in half the cases. In the second, Y copies X and becomes zero. The same observational distribution can be compatible with different intervention effects.

Causal formalism centers on this separation between conditioning, intervention and counterfactual questions [Pearl2018]. For MRP, the practical consequence is that an interpreter trained on correlations does not automatically gain authority to predict the effects of any action. It needs causal assumptions or suitable experimental data.

A PSR of a controlled system can predict action tests under its identification and coverage conditions. This does not amount to solving every causal question about hidden mechanisms or unobserved counterfactual worlds. Exactly which interventions were available and which prediction class is supported must be specified.

## Identifiability and Gaps

A model is identifiable for a question if the available data and assumptions determine the relevant answer. Two models explaining every observation may still differ regarding an untested action. In this situation, refining the internal representation does not itself produce new information about the world.

The agent must choose between experimentation, explicitly assuming a model or abstaining. Generating an additional explanation is not a fourth route to acquiring information. This limit prevents MRP from becoming a rhetoric of omniscience through internal reorganization.

## Role in the Architecture

Predictive interpreters can supply verifiable coordinates, detect collisions and suggest tests that would separate candidate models. A metacontroller can decide whether the information's benefit justifies the experiment. A verifier can check whether the final claim exceeds the domain supported by the data.

MRP can thus connect representation with action without reducing every form of understanding to next-symbol prediction. The relevant prediction may concern a protocol, an intervention, a documentary answer or a rule's consequence. Specifying this target is part of the problem, not a detail that can remain implicit.

CHAPTER 08

# Abstract Interpretations

## Sufficient Information for a Property

Abstract interpretation constructs sound approximations of program behavior. Instead of executing every concrete case, it operates in a domain preserving certain properties and ignoring others [Cousot1977]. For MRP, this provides a mature example of multiple interpreters with clear mathematical obligations.

Suppose two variables x and y both equal the same number a, and a lies between zero and ten. An interval representation preserves x between zero and ten and y between zero and ten. Applied independently to both intervals, subtraction produces the interval from minus ten to ten. Yet the concrete result x − y is always zero.

The problem is not that the intervals were calculated incorrectly. They lost the relation between the variables. A relational interpretation preserving the equality x = y can immediately prove the result. The choice of abstract domain determines which properties become accessible to analysis.

## Sound Approximation

In overapproximation analysis, the abstract result must include every possible concrete result. It may also include possibilities that do not occur in reality. These produce false alarms, not necessarily analysis errors. An approximation excluding a possible concrete behavior may become unsound for the property pursued.

We can view an abstract value as a description of a set of concrete states. “x is positive” describes many values. An abstract transformation is sound if, starting from all states described by the input, no possible concrete state after the operation is lost from the output description.

This obligation is much stronger than “the operation was correct on tested examples.” An empirically learned interpreter does not become a sound abstract analyzer merely by producing intervals or formulas. The relationship between the concrete operation and abstract transformation must be verified.

## Precision and Cost

More expressive domains can prove properties simpler domains lose, but may require more computation. Some relations matter in one program region and are useless in another. A reasonable strategy starts simply and increases precision where the required property demands it.

Cousot's introduction to abstract interpretation explains this relationship between the properties pursued, approximation and computable analysis [CousotIntro]. The MRP extension would be a controller learning which representation change is likely to resolve a particular alarm, without assuming that “more precision” is always free or necessary.

For example, the interval may suffice to verify that x does not exceed ten. To verify the equality x − y = 0, the relation is decisive. The pragmatic choice depends on the question; each interpretation's soundness depends on its rules.

## Fixed Points and Stopping

Programs with loops may require repeated information propagation. A fixed point is an analysis state unchanged by another application of the propagation rule. Not every process reaches a fixed point quickly, and some domains allow infinite chains of increasingly precise or increasingly broad approximations.

Abstract interpretation uses acceleration mechanisms, including widening, to force certain analyses to stabilize [CousotIntro]. The price may be lost precision. For MRP, the general idea is that a stopping policy must form part of the algorithm. “Keep refining until you understand” does not define a procedure with finite resources.

A prototype can initially avoid complicated domains and use finite tables or explicit step limits. The restriction is legitimate if the experiment's domain declares it. It becomes problematic only when conclusions are tacitly extended to systems without those limits.

## Two Forms of Evidence

A formal result may say: under these assumptions about the program and arithmetic, the property holds for every modeled execution. An empirical result may say: on this test distribution, the interpreter had a particular error rate. Both are useful, but they are not interchangeable.

A hybrid architecture may allow a generator to propose an abstraction and a verifier to check its local soundness. If verification fails, the hypothesis must not be promoted to a sound rule. If verification is incomplete, the result's status must preserve that limit.

## Transfer to MRP

Abstract interpretation shows that “multiple modes of interpretation” is not a new idea in itself. It also shows how this plurality can be disciplined through formal relations. MRP merits investigation where it adds a measurable mechanism for selecting, constructing and reusing these modes.

The first plausible application is not an agent inventing its own mathematics without control. It is an agent identifying which relation an analysis lacks, proposing a suitable interpreter and proving or testing exactly which improvement it achieved. This object is more modest, but difficult and useful enough for a real research program.

CHAPTER 09

# Diagnosing Errors

## Why Not Every Failure Requires a New Representation

An agent answers incorrectly. What must change? If every error leads to building an interpreter, the library quickly becomes a collection of exceptions. If no error can modify representations, the agent remains trapped within the distinctions originally designed. Diagnosis is the link between these extremes.

In the proposed model, an incident is preserved with its input, the representation actually used, interpreter version, prediction, observed result and observation conditions. The difference between prediction and result does not establish the cause by itself. It opens a bounded investigation.

| **Possible Cause** | **Example** | **Appropriate Response** |
|---|---|---|
| Computation Error | The order is available, but selection of the last event is faulty | Correct the implementation |
| Representational Loss | Different histories become the same set of events | Refine the representation |
| Unobserved Information | The revocation was not transmitted to the agent | Obtain a new observation or abstain |
| Random Variation | An operation succeeds only probabilistically | Estimate the distribution |
| Rule Change | A new version changes event precedence | Detect the change and manage versions |
| Ambiguous Specification | “Access” may mean a formal entitlement or an active connection | Clarify the target |

These causes can coexist. A faulty interpreter can also conceal missing information. Diagnosis must therefore remain revisable, rather than become a permanent label assigned by a classifier.

## Collision as a Witness of Insufficiency

A relevant collision occurs when the same representation corresponds to incompatible answers under the same test and conditions. Identifying it requires checking precisely those conditions. It is not a collision if one example uses a different rule or the questions differ.

In a deterministic microworld, two histories can supply an exact witness. In a stochastic environment, two different results are insufficient: the same distribution can produce both. There, a difference between distributions must be proved or estimated, with enough repetitions and control of relevant conditions.

A practical procedure would first seek collisions in the execution log. It would then try to reproduce the pair in a controlled environment. If reproduction confirms the difference, the agent would generate hypotheses about lost information. Otherwise, the incident remains open or is reclassified.

## Counterexample-Guided Refinement

CEGAR, counterexample-guided abstraction refinement, supplies an important discipline: an abstraction is verified and an abstract counterexample is compared with concrete behavior. If that counterexample is spurious, the abstraction can be refined to eliminate it [Clarke2000].

MRP would adopt the loop's idea without claiming that every prediction error is the same mathematical object. In verification, a false alarm may require more precision. In learning, a false prediction may require new data, a different distribution or a program correction. Identifying the failure type precedes choosing a refinement.

A proposed refinement must explain why the current representation confuses the cases. “Add another component” is insufficient explanation. “Preserve the last operation for each person–resource pair because operation order changes the answer” is an operational hypothesis.

## Minimal and General Distinctions

The program “recognize exactly the history that failed” separates any memorized case. Usually, however, it is not the distinction sought. A program tracking the last relevant operation can separate an entire family of cases. The difference between the two appears on new examples, not the initial incident.

Proposing a distinction must therefore be followed by generating contrastive cases. Change the person's identity, add irrelevant events, repeat an operation or interleave other resources. The program should preserve what matters and ignore variations irrelevant to the rule studied.

These transformations are not arbitrary. If repeating an operation changes the protocol's meaning, it cannot be treated as an invariance. The family of accepted transformations forms part of the test specification. Even constructing tests can introduce mistaken assumptions.

## Information That Cannot Be Recovered

Suppose the history has been discarded and only an unordered set remains. If two valid histories produce the same set, no program reading only that set can reliably reconstruct which history occurred. We can estimate a probability from frequencies, but this does not recover the actual event.

MRP must be able to respond: “I identified the necessary distinction, but no longer have the information needed to compute it.” This state is a useful result. It can prompt a change in memory policy or a request for a new event without fabricating a retrospective solution.

In an implementable system, diagnosis would produce one of three main outputs: a repair computable from existing information, an experiment that can supply missing information or an explicit limit. Promoting every incident into the first category would turn the architecture into a generator of unverifiable explanations.

## What Can Be Measured

Diagnostic quality has its own test set. In microworlds, we can separately inject implementation errors, collisions, missing observations and rule changes. We measure how often the agent identifies the appropriate mechanism and how much the investigation costs.

This test differs from final accuracy. A system can sometimes reach the correct answer through the wrong repair, which will fail later. For MRP, what matters is whether the improvement addresses the actual cause of difficulty, not merely whether a one-off score rises.

CHAPTER 10

# Interpreter Induction

## What Is Learned and What Is Supplied

An initial MRP system need not invent machine language, logic or every basic operation. It can receive a function core and learn new compositions. Learning's contribution must be described precisely: which primitives existed, which structures could be expressed and which programs were discovered.

For event histories, primitives may include filtering by a field, comparing two values, selecting an extremum, counting, joining by a key and applying a transition rule. A type limits admissible inputs: selecting the last record takes an orderable sequence, not a probability or an isolated proposition.

Types reduce search and prevent some errors, but do not establish intent. A program can be well typed and select the first record instead of the last. Structural correctness is a necessary condition for certain operations, not a certificate for the entire solution.

## Searching a Finitely Described Language

Induction can begin by enumerating short compositions. If there are roughly b choices at each position and length d is allowed, the number of candidate sequences may grow on the order of b to the power d. Types, symmetries and examples eliminate some combinations without removing the combinatorial difficulty.

A guided generator can prioritize programs that seem appropriate to the incident. For example, the difference between two histories with identical elements in different orders suggests order-sensitive operations. This heuristic does not guarantee success; it offers a verifiable way to reduce computation.

An LLM can participate as a generator of such hypotheses. The potential advantage is rapidly proposing compositions that would be expensive to enumerate. The disadvantage is that it may supply additional information from training, produce fragile programs and dominate the entire system's contribution. Evaluation must include a variant using the same LLM, tools and budget but lacking MRP's consolidation mechanism.

## A Selection Criterion Beyond Fitting Examples

A selection function can combine error, description complexity and execution cost. No weights are universally correct. They express the trade-off chosen for the application and must be published with the results.

Score(candidate) = estimated error + description penalty + estimated cost.

The minimum description length principle, MDL, provides a foundation for preferring an explanation that compresses both the model and the data left unexplained [Grunwald2004]. In MRP, this preference could favor a reusable rule over a list of exceptions. However, the result depends on the language in which description length is measured and on data quality. A short program is not automatically true or ethically preferable.

Separating description cost from execution cost matters. A very short definition may hide enormous search. Conversely, a longer program may precompile operations and run quickly. The library must be evaluated in terms of total cost over relevant tasks.

## From a Local Program to a Reusable Concept

Suppose the agent discovers the same pattern across several protocols: group by identity, select the last event and interpret its type. Consolidation extracts the common part and parameterizes it with the protocol's specific fields and rules.

This pattern is a candidate for an operational notion of a concept: a composition organizing many cases that can be executed in new contexts. We do not claim that every human concept is such a program. We define a kind of knowledge that can be built and measured.

DreamCoder shows that learning libraries of programmatic abstractions and guiding search can be combined in a concrete system [Ellis2020]. For MRP, the distinct question would be whether diagnosing a collision can guide this induction and whether the new abstraction explicitly improves the family of distinctions available to the agent.

## Program Equivalence and Rewriting

The library may contain different programs computing the same result. Eliminating redundancy would save memory and simplify selection. But establishing equivalence for general programs is neither free nor universally decidable [Barak].

We can work with identities proved in a restricted language. Equality saturation, including structures such as those used by egg, enables compact representation of many equivalent expressions and selection by cost [Willsey2021]. This does not authorize declaring arbitrary LLM-generated programs equivalent.

If operations have effects, arithmetic overflows or order matters, rewrite rules must include these conditions. An algebraic identity between ideal expressions may not hold for concrete execution. In a prototype, a pure and explicitly restricted core would greatly simplify verification.

## Independent Validation and Contamination

The case that generated the hypothesis cannot be the main evidence that it generalizes. Nor does a test set consulted repeatedly until the candidate passes remain an independent evaluation. Research on adaptive data analysis shows why reusing evaluation results can compromise generalization [Dwork2015].

I would separate induction examples, development tests and a final set inaccessible to selection. An evaluator could return only the verdict needed for the current stage while retaining a reserve for final evaluation. All relevant attempts should also be reported, not just the last candidate.

Successful induction does not mean “a program was generated.” It means a program constructed from declared primitives repaired an identified insufficiency, passed appropriate evaluations and justified the cost of consolidation. This is the complete experimental object.

CHAPTER 11

# The Rationality of Computation

## Computing Is Also an Action

An agent chooses not only between actions in the world but also between internal operations. It may consult a table, execute a more precise interpreter, request a new observation or construct a hypothesis. Every option consumes time and may change the final decision.

The value of computation is the expected gain from an internal operation after subtracting its cost. For a single operation followed by a decision, we can write, without assuming a perfect solution:

ValueOfComputation = expected decision gain after computation − decision gain now − computation cost.

This formula is not a new MRP law. Metareasoning and learning policies for selecting computations already have experimental formulations, including the work of Callaway and colleagues [Callaway2018]. MRP would expand the repertoire of selectable operations to include constructing and consolidating interpreters.

## A Computation Worth Performing

Assume a hidden state with two equally probable values. A correct answer is worth one unit and an incorrect answer zero. Without information, the best probability of success is 0.5. A perfect test costs 0.1 units on the same utility scale.

After the test, success is certain and net value is 0.9. The gain over answering immediately is 0.4. The test is worthwhile in the given model. If it cost 0.6, executing it would reduce total utility. Obtaining the same information may be rational in one application and irrational in another.

In practice, tests are imperfect and their value is unknown. The agent must estimate the probability of outcomes and their effects on the decision. Estimating value is itself a computation and must therefore have a budget.

## Why Myopic Selection Can Fail

Consider two hidden, independent and equally probable bits. The task is to answer whether they differ. Without observations, maximum accuracy is 0.5. Learning one bit does not improve this accuracy: the other remains unknown. Learning both raises it to one.

If each observation costs 0.1, a policy evaluating only individual operations will reject both: each costs something without immediately improving the decision. A policy evaluating the pair sees a net gain of 0.3. This is a calculated example, not a simulation result.

Interpreter composition can produce precisely this complementarity. One operation identifies entities and another relations; separately, neither answers the question. An MRP controller should therefore be tested with different planning horizons. A local rule must not be assumed to resolve every dependency.

## Avoiding Meta-Regress

If every choice requires another choice about how to choose, the architecture never stops. The engineering solution is not an infinitely wise interpreter but a bounded controller: a maximum number of evaluations, an approximate policy and a stopping rule.

Initially, selection may combine precondition checking with an empirical estimate of success on task families. Later, the policy can be learned. Even then, it must be compared with a simple rule. Controller complexity is justified only if it produces a measurable advantage.

Recent research also applies the rationality of metacomputation to language models [DeSabbata2025]. Merely selecting between a quick answer and an expensive one is therefore not MRP's distinctive contribution. The candidate contribution is the connection between selection, representation diagnosis and the accumulation of new procedures.

## The Reuse Horizon

Building an interpreter may be uneconomical for the current task but economical for many future tasks. This introduces an amortization horizon. Estimated value depends on how often the problem will recur, rule stability and verification cost.

A hypothetical example: induction and verification together cost 1,000 milliseconds. The old solution costs 40 milliseconds per task. The new one costs eight, plus two for selection. Savings are 30 milliseconds per use. The investment pays off after 34 uses if all other conditions remain identical.

It is insufficient to report that the new interpreter executes five times faster. Construction cost, reuse rate and cases where rule changes invalidate it must be included. A library's advantage concerns its lifetime, not merely its most recent call.

## Normative Limits of Optimization

A computation policy can maximize the utility supplied, but does not decide by itself which objectives are legitimate. The cost of an error to a person cannot be tacitly introduced as a number chosen by the developer. Some applications require constraints that cannot be changed merely to improve an average score.

MRP should distinguish the technical objective, approved limits and preferences still needing clarification. In a context with important consequences, abstention and human intervention may be valid actions. A meta-rational architecture does not become responsible merely by rigorously optimizing its own criterion.

CHAPTER 12

# Memory and Transfer

## Three Memories with Different Roles

The proposed architecture would separate episodic memory, representation memory and the interpreter library. Episodes preserve what was observed. Representations preserve the results of transformations. The library preserves procedures that can reconstruct, query or modify those results.

This separation does not require three physical databases. It requires being able to answer three questions: “which information was available?”, “how was it compressed?” and “which operation produced this conclusion?” Without these answers, a representation error can easily be confused with an observation error.

Complete raw memory is often expensive or inappropriate. Yet irreversible compression limits reinterpretation. MRP should treat retention policy as an explicit choice, with classes of recoverable and unrecoverable information, rather than an infrastructure detail.

## Provenance Without Excessive Promises

A result should be linked to its input version, interpreter, parameters and the activity that produced it. W3C's PROV model supplies a vocabulary for entities, activities, agents and derivations [W3C2013]. It can inform log interoperability without requiring every internal component to use the same representation.

Provenance does not guarantee truth. A false claim may have a perfectly documented history. What we gain is the ability to inspect dependencies, attribute transformations and determine which results must be reconsidered after a change.

Two documents repeating the same source do not automatically constitute two independent pieces of evidence. A provenance library can reveal their common dependency. Confidence aggregation must account for it, rather than simply count references.

## Transfer as a Test of Abstraction

There are at least three levels of reuse. Rerunning the same program on different names is a modest level. Reusing a composition on a new structure with different rule combinations is stronger. Transfer to a domain with different observations and validity conditions is harder still.

An interpreter selecting the last operation may work for both a permission and an updatable preference. It does not follow that it works for a bank balance, where all relevant operations may contribute cumulatively. Superficial similarity between logs does not justify transferring the rule.

An application contract should specify this difference: is the property determined by the last operation or by aggregating operations? This is itself a possible meta-level of learning. The agent can learn which interpreter family suits a class of processes.

## Consolidation, Forgetting and Migration

Adding interpreters increases capability but also search cost. Consolidation can merge redundant definitions, introduce parameters and retire inferior variants. Retirement must not erase the history of results depending on them. An old version may remain necessary to reproduce an analysis.

A retention criterion can combine recent use, estimated value, uniqueness of proven properties and reconstruction cost. This criterion must be tested for catastrophic forgetting: losing a rare but important capability is not automatically justified by its low frequency.

Migrating a representation requires an explicit transformation. If the new interpreter needs event order and the old memory does not preserve it, migration cannot be complete. The system must mark reconstructed, estimated and unknown segments.

## Long-Term Learning

An adaptive library's performance must be tracked over time. A learning curve can measure average cost per task as the library grows. A retention curve measures whether old tasks remain solvable. A transfer curve measures advantages on new families.

These curves may have different shapes. The agent may become faster on familiar tasks but weaker on new ones because the selector favors routine excessively. Or it may generalize better while consuming unsustainable memory. MRP must be evaluated as a dynamic system, not through a single number after training.

On this view, accumulated knowledge would be more than a repository of propositions. It would be a collection of procedures, results, conditions and evidence enabling the agent to reconstruct a distinction when it becomes relevant. The usefulness of this organization remains an empirical hypothesis, but its components are precise enough to implement.

CHAPTER 13

# Pragmatic Distances

## Closeness Relative to a Question

Two descriptions can use very similar words yet require opposite actions. “Permitted until Friday” and “prohibited until Friday” are lexically close but different for an access decision. Conversely, very different formulations can describe the same rule. A distance between texts is not automatically a distance between their consequences.

For MRP, the starting question is: how different are the results of relevant tests for two histories? This establishes both the distance's purpose and its limits. We are not defining a universal similarity of things, but similarity relative to a declared family of experiments.

## Total Variation

For two discrete distributions P and Q over the same outcomes, total variation distance is half the sum of absolute differences between their probabilities. It is also the greatest difference between probabilities assigned to the same event.

TV(P, Q) = 0.5 × the sum, over outcomes, of |P(outcome) − Q(outcome)|.

If P and Q are distributions of a binary answer, the distance is the absolute difference between the probabilities of answering “yes.” Predictions of 0.2 and 0.7 are at distance 0.5. If they are identical for every outcome, the distance is zero.

Now fix a test family T. For each test, the two histories induce two distributions over the outcome. The proposed pragmatic distance is the greatest difference between these distributions across the family's tests.

dT(h1, h2) = the supremum, for t in T, of TV(Pt(. | h1), Pt(. | h2)).

The supremum is the least upper bound. For a finite test family, it is simply the maximum. If a test includes actions, the distributions concern executing that test within the declared model, not observational conditionings tacitly interpreted as interventions.

## Why It Is a Pseudometric

The distance is nonnegative and symmetric. It is zero between a history and itself. The triangle inequality follows from the corresponding inequality for each test, followed by taking the supremum. The mathematical appendix provides a step-by-step proof.

However, distinct histories can have zero distance: family T may not distinguish them. We therefore call the function a pseudometric. Grouping all histories at zero distance yields a metric on the resulting classes. This construction connects distance with the equivalences in chapter 5.

Behavioral metrics and bisimulation-based distances have important precedents in the study of decision processes [Ferns2004]. The distance formulated here must not be confused with all those metrics: the choice of tests, rewards and horizon changes the mathematical object.

## Approximation Does Not Automatically Produce Classes

In practice, we often require not exactly zero distance but a difference below a threshold. A trap appears: the relation “distance is at most the threshold” is not generally transitive.

For three binary distributions with probabilities 0.00, 0.06 and 0.12, the first two are close at a threshold of 0.08, as are the last two. The first and last are not. If an algorithm merges every close pair through chains, it can produce a class whose members differ far more than the original threshold.

Approximate clustering therefore requires an additional criterion: maximum class diameter, a center with a controlled radius or an explicit bound on aggregation error. The statement “we represent all similar states through the same concept” is incomplete without this criterion.

## When Distance Controls the Decision

Suppose a test's outcome determines a utility between zero and one. The difference between mean utilities under P and Q is at most TV(P, Q). A small predictive distance can thus bound error in a decision evaluation, provided utility is actually calculated from the modeled outcomes.

A second simple result is useful. If every action's estimated value is within e of its true value and we choose the action with the highest estimate, the loss relative to the genuinely optimal action is at most 2e. One e term comes from underestimating the optimal action and the other from overestimating the chosen one.

These bounds do not automatically transfer from a single step to every long plan. For sequential actions, the relevant trajectory distributions or error propagation must be controlled. Good next-event prediction does not by itself guarantee a good long-term policy.

## Estimating Distance Has a Cost

Exact distance assumes knowledge of the distributions. Finite data yield estimates and uncertainty. An agent declaring two histories identical because it has not yet found differences may confuse absence of evidence with proof of equivalence.

For the first experiment, I would use microworlds where the evaluator knows the true distributions while the agent receives only samples. This allows us to measure how well the agent estimates closeness and when it declares a representation sufficient prematurely.

Distance serves to turn a metaphor about the “neighborhood of meanings” into a verifiable claim: these situations have similar consequences for these tests, with this estimated error. Only afterward is it worth seeking an efficient geometric representation of these relations.

CHAPTER 14

# Cones and Coordinates

## What We Call Geometry

A geometric representation assigns coordinates to objects and uses relationships between coordinates to express relevant structure. The choice of distance, dimensions and admissible transformations forms part of the model. A collection of texts, experiences or programs has no single implicit geometry.

Gärdenfors's conceptual spaces theory proposes describing concepts through quality dimensions and geometric regions [Gardenfors2000]. It motivates questions about the shape of concepts, but does not prove that all knowledge must be convex in a fixed space. For MRP, the structure to preserve must be specified: similarity, order, behavior or decision preference.

## Convexity and Cones

A set is convex if the segment between any two of its points remains inside it. A disk and a cube are familiar examples. A ring is not convex: the segment between opposite points can cross the central hole.

A cone, in the sense used here, is closed under multiplying its coordinates by a nonnegative number. If it is also convex, nonnegative combinations of its points remain in the cone. A cone need not be imagined only as the circular object from school geometry; it can be defined by linear inequalities in many dimensions [Boyd2004].

High dimensionality does not imply nonconvexity. Convex sets exist in every finite dimension. A cone can also be convex. Opposing “cones” to “convex shapes” would mix two different classifications.

## A Decision-Theoretic Derivation

Assume finitely many possible world states. The agent has a distribution p over them. Each option i is a fully specified procedure: executing an interpreter, followed by a fixed rule for choosing an action from the result. For each state, the procedure has net utility u_i, including the modeled costs.

The word “fixed” is essential. If the internal procedure changes arbitrarily with p, its utility is generally no longer represented by a single constant vector. We can include multiple candidate procedures, but cannot retain the linear derivation while ignoring that change.

The mean utility of procedure i is the dot product of p and u_i: the sum of each state's probability multiplied by utility in that state. Procedure i is preferred to procedure j when this product is at least as large.

Region(i) = {p: every coordinate is nonnegative, their sum is 1, and $p \cdot (u_i - u_j) \geq 0$ for every j}.

Each comparison defines a half-space. Their intersection with the probability simplex is a convex polyhedral region. The simplex is the set of all distributions over the listed states: a segment for two states, a triangle for three and a higher-dimensional analogue in general.

We now move to unnormalized weights z, all nonnegative. The preference conditions become homogeneous: multiplying every weight by the same positive number does not change the preferred procedure.

Cone(i) = {z: $z \geq 0$ and $z \cdot (u_i - u_j) \geq 0$ for every j}.

This set is a convex polyhedral cone. The original region is obtained by intersecting it with the plane where the weights sum to one. The origin is mathematically included in the cone but does not represent a normalizable distribution.

## A Calculated Example

We have two possible states and three procedures. The first has utilities (1; 0), the second (0; 1), the third (0.6; 0.6). These hypothetical values include every assumed cost. Let p denote the probability of the first state.

The first procedure has value p, the second 1 − p and the third 0.6. The first is optimal for p above 0.6; the second for p below 0.4; the third for p between 0.4 and 0.6. Options tie at the boundaries.

In unnormalized coordinates, the third procedure is preferred when each weight is at least two-thirds of the other. We obtain a sector in the first quadrant. The cone describes a family of informational states in which selecting the same procedure is rational.

This gives the geometric intuition a concrete meaning: some “regions of understanding” may be regions of decision applicability. It does not follow that every concept, proposition or memory is a cone.

## Entailment Cones and Other Meanings

Ganea and colleagues use entailment cones, including in hyperbolic geometry, to represent hierarchical relations [Ganea2018]. There, the construction is motivated by preserving a partial order. In our derivation, it is motivated by comparing expected utilities.

These two types of cones may be useful in an MRP system, but they are not interchangeable. A subordinate concept in a taxonomy is not the same as an optimal procedure in a region of beliefs. Using the same drawing does not establish identical mechanisms.

## What Happens to Rings and "Holes"

Consider the region between two circles: points satisfying $1 < x^2 + y^2 < 4$. It is nonconvex. An interpreter computing $r = x^2 + y^2$ transforms the question "does the point belong to the region?" into checking the interval $1 < r < 4$.

The transformation loses the angle, however. Different points are compressed into the same value. We have not freely eliminated geometric complexity through an invertible coordinate change; we have discarded information unnecessary to the radial question. For a question about direction, the new representation would be insufficient.

Holes and connectivity have a different status from convexity when only continuous transformations with continuous inverses are allowed. But MRP interpreters may be discontinuous and compress information. Intuitions about rings or tori must therefore be linked to an explicit class of admissible transformations.

## The Testable Hypothesis

I would not say "knowledge has the shape of cones." I would say: for this task family, a representation using particular geometric regions preserves relevant distinctions and enables more efficient selection or transfer than alternatives under the same budget.

The test would compare applicability errors, search cost, memory and transfer. A beautiful visualization is not evidence. If a simple table achieves the same results at lower cost, the geometric hypothesis has not yet produced engineering value.

CHAPTER 15

# An Atlas of Interpreters

## Why Not a Single Space

The same situation may require preserving temporal order, participant identity, a numerical relationship or a claim's provenance. A single space can encode all of these in principle, but accessing each property may remain expensive or difficult to verify.

The atlas proposal is more modest: multiple local representations, each with well-defined operations and a domain of use. “Atlas” is an organizational analogy. It does not automatically imply a differentiable manifold, smooth coordinates or invertible transformations between every representation.

A temporal interpreter may answer questions about the latest state. A relational interpreter may answer questions about dependencies. A documentary interpreter may preserve who supports each claim. They can describe the same episode without producing the same kind of result.

## Translations with Contracts

A translation between two representations must declare what it preserves. If we construct a current state from a log, we may preserve the answer to “what is active now?” but not necessarily “how often did it change?” A useful translation need not be fully reversible; it must be sufficient for its stated purpose.

In an atlas, links would be executable adapters. Each adapter has preconditions, an output type and a claim about information preservation. Some preservation claims may be proved. Others can only be evaluated through tests. Their status must be retained in metadata and the explanation of the result.

A round-trip test, translating A into B and back again, is useful for detecting losses. Success on a few examples does not prove equivalence, however. Moreover, an intentionally compressive translation should be evaluated by preservation of relevant tests, not complete reconstruction.

## Local Compatibility and Contradiction

Two representations may produce apparently contradictory results without being logically incompatible. One describes the formal rule, the other the observed state. A person may have access rights while the connection is unavailable. The contradiction disappears when targets are typed correctly.

In other cases, the contradiction is real: two interpreters assert different values for the same variable, period and assumptions. The system must not arbitrarily average them. It must check sources, preconditions, versions and dependencies.

This work can be represented as a constraint satisfaction problem. If a set of claims cannot all be true in the declared model, the agent seeks a conflicting subset and marks it. A solver can help for suitable languages without automatically resolving the natural-language ambiguity that produced the formulas.

## Operations Define Coordinates

An MRP atlas would store not only values but also the procedures producing them. The coordinate “latest valid state” is a program over a history. “Number of independent sources” is a procedure over a provenance graph. “Estimated risk” is a model with its own data and assumptions.

This connection allows a new dimension to be checked. We can inspect the program, modify input data and observe what changes. A learned numerical dimension without a simple interpretation is not excluded, but does not automatically receive the same kind of justification.

Here lies one of the program's central ideas: a new dimension of understanding can be a procedure producing a previously inaccessible distinction. The resulting geometry depends on the procedure and the questions for which we use it.

## Evaluating the Atlas

For every task, we can measure how many representations were consulted, how many translations were executed, what information was lost and what reconciliation cost. The atlas must be compared with a sufficiently powerful single space and a fixed library of views.

A favorable result would show local adaptation avoiding reconstruction of the entire representation and transferring procedures between tasks. An unfavorable result would show adapters, selection and checks costing more than a well-designed common model. Neither conclusion should be presumed from the chosen metaphor.

The atlas is not an architectural obligation. It is an organizational hypothesis that can be implemented gradually. An initial system may have only three views and two adapters; what matters is demonstrating what each preserves and when changing views resolves a real difficulty.

CHAPTER 16

# The Architecture of an MRP Technology

## A Functional Architecture, Not an Assumed Platform

We can describe an MRP technology without confusing it with a product already built. It should receive a task, establish which information is needed, select and execute interpreters, verify results and learn from identified insufficiencies. Every verb must correspond to an observable object and transition.

In this book, MRP-VM designates a possible execution machine for these functions. “Virtual machine” does not necessarily mean a new processor or bytecode. It means an environment controlling the state, calls, dependencies and acceptance conditions of interpretive operations.

A minimal implementation can use an ordinary database, a graph scheduler and existing executors. Its value would come not from the number of technologies introduced, but from linking an observed problem to a verifiable change in the repertoire.

## Persistent Objects

| Object | Minimum Content | Role |
|---|---|---|
| Task | Question, inputs, success criterion, budget | Defines what must be solved |
| Episode | Ordered observations, identities, time and sources | Enables reinterpretation |
| Representation | Values, schema, constructor program, version | Preserves distinctions |

| | | |
|---|---|---|
| Interpreter | Contract, implementation, dependencies, cost | Transforms and verifies |
| Evidence | Test or proof, assumptions, result, artifacts | Supports a precise status |
| Incident | Prediction, observation, difference and diagnosis | Triggers adaptation |
| Library | Approved interpreters and application criteria | Enables reuse |

These objects must have stable identities. A result cannot point merely to "the temporal interpreter," because its definition can change. It must identify the version actually executed and the exact inputs or their verifiable identity.

A cryptographic content identifier can help link artifacts with evidence. It does not establish the result's truth or replace program semantics. Its role is narrower: detecting changes in the content to which verification refers.

## An Interpreter's Contract

The contract must specify what the interpreter receives and can produce. For a last-operation interpreter, the input may be a sequence of events with identity and total order. The precondition may require every event to have a comparable position. The output is a state value for each key.

The contract would also declare limits: it ignores events absent from the input; assumes the "last operation prevails" rule; and does not establish that the log is complete. A correct result about the available log is thus not presented as evidence about every real event.

For a probabilistic interpreter, the contract includes the distribution's target and evaluation conditions. For a solver, it includes the theory and model checked. For an LLM, it may declare the output a textual proposal or candidate program, rather than a certified result.

## The Controller and Executors

The controller decides which operation comes next. Executors may be symbolic interpreters, programs, statistical models, solvers, database queries or people asked to clarify a specification. A uniform interface does not imply uniform trust in the results.

An initial controller can use transparent rules: check preconditions, choose the cheapest compatible candidate, execute, evaluate and escalate if an incident occurs. This version provides a baseline for a later learned controller. Without a baseline, any additional complexity can seem useful because it has nothing to be compared with.

Executors would separately report time, memory consumption, external operations and final state. A timeout is an execution outcome, not proof that the property cannot be established. A parsing error differs from a semantic counterexample.

## The Epistemic Register

I would separate a result's content from its status. “Permission is active” is content. “Computed exactly from the complete log according to model M” is one status. “Estimated from incomplete observations” is another. The two must not be lost as the result passes through other components.

A register could distinguish reported observation, hypothesis, derived result, formal verification relative to a model, empirical validation and withdrawn result. These categories do not necessarily form a single hierarchy. A formal proof about a mistaken model and a robust observation about the real system have different limitations.

Composition should preserve relevant dependencies. A final text based on a hypothesis does not become fact merely because the last stage is a fluent generator. A verifier can confirm a document's form without confirming every claim's truth.

## The Minimal Useful Implementation

The first implementation needs no complete ecosystem. It needs a family of finite tasks, an operation core, a few initial representations, an execution log, a candidate generator and a separate evaluator. This minimum already allows collision, refinement and transfer to be tested.

Extensions should be introduced only when they address an identified limit. If selection becomes expensive, develop library indexing. If evaluation consumes too much, study test-selection strategies. If transfer does not emerge, investigate induction or task definitions before adding more components.

A defensible MRP architecture is one whose advantages can be attributed to its mechanisms. Suggestive names for every component are insufficient; components must be removable one at a time to measure their contributions.

CHAPTER 17

# Execution, Evidence and Revision

## A Candidate's Lifecycle

A new interpreter passes through distinct states. First it is a proposal. Then it is checked structurally and executed in a controlled environment. Later it may receive empirical evaluation or formal justification for certain properties. Only after meeting the established criteria is it admitted to the usable library.

| State | What Is Permitted | What Is Not Assumed |
|---|---|---|
| Proposed | Inspection, structural checking | Semantic correctness |
| Executable | Testing with limited resources | Generalization |
| Evaluated | Use within the evaluated domain, according to policy | Universal validity |
| Consolidated | Reuse and monitoring | Immunity to change |
| Suspended | Diagnosis and reproduction | Current automatic use |
| Withdrawn | Audit and reproduction of the past | Recommendation for new cases |

Proofs can accompany different states and cover only part of the contract. “Consolidated” is a library-management decision, not a synonym for “mathematically proved.” This distinction prevents an operational approval from becoming an exaggerated epistemic claim.

## A Complete Walkthrough

The task is to determine current permission from a log. The initial representation stores only which event types occurred. The agent encounters two histories with identical types but different outcomes. The evaluator confirms that order, rather than noise, explains the difference.

The generator proposes an interpreter that groups events by person and resource, selects the final position and applies the state rule. Structural checking confirms that the types match. Tests check reversed order, interleaved keys and repeated operations according to the environment's rule.

An additional verifier may prove, for the finite event language and given implementation, that the result corresponds to the last relevant operation. This proof does not confirm that the real log is complete; it confirms the relationship between input and output within the model.

After acceptance, the program enters the library. On a new task, the selector must recognize the appropriate rule. If the new protocol is cumulative, the contract does not apply. Correct transfer includes refusing to use an unsuitable abstraction.

## Epochs and Controlled Changes

Learning changes data interpretation. If an interpreter is replaced during execution, the result may mix versions without clear justification. One solution is to use epochs: each execution sees a stable version of the library and policy.

A modification produces a new epoch. Old results remain associated with the configuration in which they were obtained. The new interpreter can be tested on copies of tasks before becoming the default. This facilitates reproduction and comparison without requiring the entire system to stop whenever learning occurs.

The change can be transactional: either the interpreter, representation schema and affected dependencies are updated consistently, or the old version remains active. Transaction is used here as a consistency discipline, not a promise that every external effect can be undone.

## External Effects and Reversibility

An interpreter reading data differs from one modifying a database or sending a command. Proposed actions must be separated from actual executions. Repeating a check must not inadvertently repeat an irreversible effect.

For the research prototype, operations can be limited to a simulated environment or copies without external effects. When moving to real applications, permissions, operation idempotence, approvals and effect records must be defined. An idempotent operation has the same effect on state

when repeated as when executed once. These requirements are not a philosophical addition; they determine which experiments are reproducible.

A sandbox limits the consequences of faulty programs but does not establish that their results are correct. Conversely, a proof about a pure function does not authorize every use of it in a system with effects. These problems must be addressed separately.

## Selective Recomputation

If a source is corrected or an interpreter withdrawn, not everything needs recomputing. We can trace the dependency graph and mark affected results. This optimization assumes, however, that dependencies are recorded sufficiently completely.

A result that indirectly reads a global configuration without declaring it creates a hidden dependency. Reusing it from memory may be wrong after the configuration changes. An execution's identity therefore includes not only its visible input but also relevant context parameters and versions.

The recomputation policy can begin with an overapproximation: invalidate every possibly affected result. It can later be refined to avoid unnecessary work. The same logic of precision and cost reappears at infrastructure level.

## What an Execution Trace Explains

A good trace shows which operations ran, with which inputs and checks. It can explain the process producing the result. It does not automatically supply a causal explanation of the worldly phenomenon or a value-based justification of the chosen objective.

MRP should deliver these levels separately. A scientific result requires both reproducible computation and adequate models and data. An organizational decision also requires authority and legitimate criteria. Technical transparency is valuable precisely when it does not present itself as the solution to every one of these questions.

CHAPTER 18

# MRP-VM and SOP Lang

## The Theory Does Not Depend on a Syntax

MRP describes the organization of interpretations and their learning. SOP Lang may be an option for representing executable compositions, but the theory must not depend on a particular file extension or naming convention. Other languages can implement the same experiment.

This separation enables two evaluations. We can test whether the MRP mechanism produces transfer and efficiency. Separately, we can test whether SOP circuit representations facilitate composition, provenance tracking and verification. A syntax or parser failure does not automatically invalidate the general hypothesis.

## The Circuit as a Unit of Composition

In one possible implementation, a circuit is a graph of operations. Nodes execute transformations; links carry values and dependencies. Inputs and outputs are declared. An operation may call a primitive or another circuit, and composition produces a more complex procedure.

To simplify analysis, intermediate results can have unique identities within an execution. Instead of tacitly modifying a value, we produce a new one and record the dependency. This single-assignment discipline helps track provenance without excluding controlled persistent state.

An acyclic graph can be executed in dependency order. Recursive or iterative processes require an additional mechanism: an explicit loop, a fixed point or the opening of a new stage. Merely drawing a graph resolves neither termination nor effect management.

## A Small Core, an Extensible Library

The core would implement operations needing direct support: data access, type checking, certain computations and communication with executors. Most higher-level compositions could be declarative circuits. A new interpreter would thus become an inspectable library object, rather than an opaque platform modification.

The distinction between primitive and circuit must be visible in system documentation. Otherwise, the agent may be claimed to have learned a capability actually hidden inside a complex manually supplied operation. An evaluator must see exactly what was compiled, learned and called.

File organization can define command namespaces, but this convention is not a cognitive contribution. It becomes useful if it simplifies component discovery, versioning and reuse. MRP evaluation must remain centered on capabilities, not organizational aesthetics.

## Controlled Language and Intermediate Representation

A controlled-language interface can make circuits easier to read. For a rigorously defined fragment, we can require textual and intermediate representations to be interconvertible without semantic loss within that fragment. This is a verifiable design requirement.

We cannot extend the same promise to every natural-language sentence. Ambiguity, presuppositions and context may permit several interpretations. The translator must preserve alternatives or request clarification, rather than arbitrarily declare that it extracted the “exact meaning.”

A verification loop can compare a circuit's effects with positive and negative examples supplied by the user. These examples do not resolve every ambiguity, but they make the translation open to challenge. Stabilized meaning becomes the accepted contract for that operation.

## The Role of Language Models

An LLM can translate a request into a circuit proposal, generate tests and suggest a new distinction. It can also serve as a probabilistic executor when no suitable symbolic procedure exists. The architecture must allow these roles without confusing proposal with verification.

If the same model generates the program and every test, its errors may be correlated. Independent evaluators, specification-generated tests or human interventions are needed for important properties. Independence does not necessarily mean another model brand; it means a source of constraints that does not repeat the same unverified assumption.

Initially, a hybrid system can be very useful even if most program ideas come from an LLM. But the experimental report must say so. MRP's contribution may be verification, consolidation and reuse, rather than the initial generation of every concept.

## A Composition Example Without Artificial Syntax

For a documentary synthesis, a circuit can receive a document and a family of questions. One stage identifies candidate passages. Another separates the author's claims from quotations and hypotheses. A third constructs answers with references to passages. A verifier checks whether the summary preserves the required exceptions.

If the verifier finds a sentence in which a hypothesis became fact, the incident need not require a new text generator. It may require an interpreter preserving the modality of claims and a composition rule prohibiting its loss. This new operation can be reused in other documents.

The circuit makes the distinction visible; MRP decides when it must be constructed and whether consolidation is worthwhile. This is the functional relationship between the two levels. Neither should be presented as an already validated solution for all of natural language.

CHAPTER 19

# Software and Protocol Verification

## Why This Is a Suitable First Domain

Software offers situations in which inputs can be controlled, executions reproduced and precise properties formulated. Not every software system is simple to model, but a bounded protocol family can separate representational difficulty from natural-language ambiguity. This is therefore the first application I would prioritize.

The commercial or engineering problem would be reducing repeated analysis costs without losing required properties. An experimental MRP system would begin with simple views of the program or log and activate more precise representations only where demonstrable insufficiencies arise.

The system need not immediately compete with every existing verification tool. An initial contribution may be better selection among known analyses or learning preprocessing steps that make a property accessible to an existing analyzer.

## Use Case: The Reservation Protocol

Consider a reservation service with requests, confirmations and cancellations. The agent receives an executable specification of the protocol version and labeled logs. The question is whether the resource is reserved at a given moment and which continuations can change its state.

A coarse interpreter preserves only the existence of a confirmation. It fails after cancellation. A second preserves the last operation but may fail if the protocol distinguishes canceling a request from canceling a confirmation. The necessary distinction depends on the exact rules, not operation names.

The agent can learn a state model with preconditions. Confirmation is valid only after an active request, while certain operations are ignored or produce an error. Contrastive tests must include invalid operations, different identities and histories longer than those used during induction.

For a finite protocol, the evaluator can explore every reachable model state. If inputs and states are unbounded, exhaustive proof must be replaced or supplemented by inductive arguments and tests. The size of the verified domain must be reported, rather than hidden behind “fully tested.”

## Use Case: Choosing the Abstract Domain

In numerical program analysis, the agent begins with intervals. A property cannot be proved because the analysis loses equality between two variables. Diagnosis identifies the necessary relation and activates a relational interpreter only for the relevant portion.

There are two learning levels here. The first learns to choose among domains already implemented. The second proposes new predicates or transformations, verified before use. The first is simpler and provides an important baseline for the second.

If a generator proposes an invariant, the verifier must check initialization, preservation through transitions and whether the invariant implies the required property. Being true on observed traces is insufficient. On a finite trace set, many generally false claims may appear invariant.

Abstract interpretation and CEGAR supply the technical precedents for this discipline [Cousot1977; Clarke2000]. MRP's contribution should be sought in identifying and reusing distinctions, not renaming these methods.

## Use Case: Migrating an API

A new service version may change a validity condition or precedence rule. In a controlled experiment, the agent receives examples from both versions and must distinguish a rule change from an accidental fault.

The old library may remain useful for unchanged parts. The agent should create a new contract or version, rather than overwrite the past. Earlier results must remain reproducible relative to their version.

Value would come from reducing reanalysis work: reusing compatible interpreters, reconstructing only affected distinctions and generating tests that separate the versions. One success criterion is identifying the actual change without excessively increasing false alarms.

## What We Measure

The main measures would be correctly proved properties, missed errors, false alarms, total time, memory and the cost of constructing new interpreters. Time to the first valid counterexample may matter, but must not be used alone: a system may find one example quickly and miss other error classes.

Comparisons would include a single more precise analysis, a fixed combination of analyzers and a selector without induction. To isolate reuse, evaluation tasks must contain new structures, not copies of the same programs with renamed variables.

An informative negative result would show selection from a fixed library producing the entire advantage, with induction adding nothing after accounting for its cost. This result would favor a simpler technology. MRP would remain a useful analytical framework, but the adaptive-library hypothesis would be weakened within the tested domain.

## Adoption Conditions

I would consider the application promising where many related tasks exist, specifications are sufficiently clear and the verifier is more reliable than the generator. I would not begin with systems where the target continually changes, observability is very poor and evaluating an error costs more than manual analysis.

Initial integration should be advisory: suggest analyses, produce tests and explain which relation is missing. Only after independent evaluation could it automate decisions with effects. Immediate usefulness may consist in accelerating an engineer's work, without claiming a universal autonomous verifier.

CHAPTER 20

# Documents, Synthesis and Procedures

## The Summary as a Lossy Representation

A summary compresses. The question is not whether it loses information, but whether it loses information needed for its use. A summary for general orientation and one for applying a procedure have different criteria. The same reduction may be acceptable in the first case and dangerous in the second.

MRP would represent this requirement through a family of relevant questions and transformations. Entities, conditions, exceptions, temporality and claim status must be preserved insofar as they affect answers. We do not assume this family can cover every future question.

A documentary result would include the summary, links to source passages and a report on checks performed. This would allow the user to distinguish “I found no exception” from “I checked every relevant section of this document according to the defined procedure.”

## Use Case: A Procedure with Exceptions

Imagine an internal procedure: requests are approved automatically below a certain value, except those involving a critical resource; exceptions require additional approval. This text is an invented example, not a description of an actual legal or organizational arrangement.

A summary preserving only the threshold may produce wrong answers. Its representation confuses two requests with the same value but different resources. The counterexample indicates a conditional distinction, not necessarily a fluency problem.

A new interpreter can extract the general rule, exception and precedence relation. Another can check whether a summary sentence omits a necessary condition. For a controlled fragment, rules can become executable predicates. For ambiguous texts, the system must retain candidate interpretations and mark the unresolved question.

Tests would vary request value, resource type and the presence of approval. Evaluation cases would include differently worded exceptions and unseen combinations, not merely copied uses of “except.” This separates lexical recognition from preservation of the logical relation.

## Use Case: Claims and Attribution

A report says a team supports hypothesis H but the results do not yet confirm it. A faulty summary may state H directly. The lost information is not a numerical datum but modality and attribution: who supports what and with which status.

A documentary circuit could construct records containing the claim's content, attributed author, hypothetical or asserted character, source passage and relation to evidence. This representation does not guarantee correct extraction, but allows these dimensions to be tested separately.

A verifier would compare the summary's claims with accepted records. If a claim's status is stronger than the source's, it would flag an unjustified promotion. This kind of distinction is reusable across documents and may be a good target for library learning.

## Two Links That Must Not Be Confused

The first link is translating text into a representation. The second is computing over that representation. A solver can impeccably prove that a conclusion follows from extracted rules while the extraction itself is wrong. Evaluation must measure both links.

For an initial experiment, we can generate documents from known structures and vary their expression. This supplies an exact reference for the relations studied. Later, authentic documents must be used with independent annotation and room for evaluator disagreement.

Synthetic data help isolate the mechanism, but may remain too close to the generator's language. Success on them does not automatically establish usefulness on real documents. The two stages must be reported separately.

## Quality Criteria

Accuracy on relevant questions is one criterion, not the only one. Coverage, exception preservation, faithful attribution, source localization and justified abstention also matter. A very short summary may correctly answer three known questions yet become useless for the rest of the task.

Some evaluation questions must be hidden from the generator. Human assessment of usefulness must also be included without confusing it with stylistic preference. A reader may prefer a fluent text that omits precisely the important warning.

Cost must be compared with a reasonable baseline: an LLM with source access, search and similar checks. If MRP receives an additional verifier and memory denied to the alternative, the experiment does not isolate the architecture.

## Usefulness and Limits

The application could be valuable for technical manuals, product documentation, internal procedures and research syntheses. Interpreter libraries would preserve recurring distinctions: condition versus conclusion, hypothesis versus reported fact, current state versus history, primary source versus indirect citation.

In domains with legal or medical consequences, this architecture should be studied as documentary support under specialist oversight, rather than presented as an autonomous authority. A proof about a summary's structure is not professional validation of its recommendations.

The intended advantage is reducing identifiable errors and verification costs across repeated documents. It is not a promise that every natural meaning can be fully formalized. A useful technology can have explicit limits and solve only certain types of distinctions very well.

CHAPTER 21

# Research and Discovery

## Three Different Activities

A research agent can generate hypotheses, select experiments and evaluate results. These activities are not identical. An interesting hypothesis may be false; a well-chosen experiment may yield a negative result; a correct analysis may lead to no new discovery.

MRP could organize their relationship through interpreters with different roles. Some construct candidate models. Others calculate predictions. Others identify tests separating models or check whether a conclusion exceeds the data. Each role's success must be evaluated against its own target.

I would not define novelty through a model's internal surprise. A claim may be rare in the agent's representation and well known in the literature. Conversely, a new idea may be expressed in very ordinary terms. Evaluating scientific priority requires external sources and a delineated contribution.

## Use Case: Two Mechanisms Compatible with the Data

Assume a simulated system in which two models explain the same passive observations. In the first, X and Y have a common cause. In the second, X influences Y. The example in chapter 7 shows that observing X and Y may not separate these mechanisms, while an intervention can.

A predictive interpreter identifies the models' agreement on existing data. An intervention interpreter calculates which experiments produce different outcomes. The metacontroller compares experiment costs with the value of separation. After observation, the model library is updated.

In a microworld, the evaluator knows the real mechanism. We can measure how many experiments identification requires and their total cost. In a real laboratory, assumptions about interventions, instruments and errors become part of the problem. The observation–intervention distinction is fundamental, but does not replace experimental design [Pearl2018].

## Learning a Discovery Procedure

A more interesting contribution than solving one case would be learning a reusable procedure. For example, when two models differ only in one relation, the agent seeks an intervention that blocks the alternative explanation. The procedure must be formulated under conditions where this reasoning is valid.

The same pattern cannot be mechanically transferred to every system. The intervention may change other variables, be impossible or have delayed effects. A reusable interpreter should request these conditions rather than assume them.

The research library would therefore store both experiment templates and their preconditions. Learning would mean not merely "which experiment worked," but "in which class of situations does that experiment identify the relevant difference?"

## Compression, Explanation and Discovery

A shorter model may compress observations. This is useful but insufficient for causal explanation. An interpolating formula may predict existing examples without capturing intervention mechanisms. A reusable concept must be evaluated on new consequences.

I would require three distinct tests for a hypothesis: fit to known data, predictions under unseen conditions and, where causal meaning is claimed, predictions about appropriate interventions. Absence of the third test does not invalidate a predictive model, but limits its interpretation.

MRP can make these limits visible in the epistemic register. A result can be an excellent predictor while remaining unsupported as a mechanistic explanation. This separation helps researchers precisely because it prevents a tacit leap from performance to ontology.

## Use Case: Reviewing a Literature

The agent receives a bounded corpus of articles and a research question. One interpreter extracts relevant claims, another identifies study methods and conditions, and another separates independent confirmations from citations of the same evidence.

An incident occurs when two results appear incompatible. The agent may discover differences in populations, measurements or definitions. The new distinction does not automatically resolve the dispute, but may reformulate the question so the conflict becomes investigable.

Evaluation would measure attribution correctness, identification of conditions and the usefulness of proposed questions for experts. I would not use the number of self-declared “new ideas” as the main metric. It is too easy to increase through rewording or unverifiable hypotheses.

## The Relationship with the Human Researcher

The researcher establishes important problems, ethical limits and relevance criteria. The agent can help identify overlooked distinctions and explore their consequences. This assumes no rigid division between human creativity and automated computation; it requires explicit responsibilities and evidence.

An initial system should produce hypothesis dossiers: model, assumptions, data used, discriminating predictions, proposed experiments and reasons the result would matter. Its value can be evaluated before any spectacular discovery through time saved and methodological errors detected.

The application is ambitious but can be built gradually. The first realistic target is an agent making research more verifiable and choosing experiments better within a narrow domain. It is not a machine guaranteeing novelty or replacing scientific-community validation.

CHAPTER 22

# Education and Interactive Agents

## Education as Diagnosing Distinctions

A pupil or student may apply a rule to ordinary examples yet confuse two situations requiring different treatment. An MRP educational system would try to identify this collision in responses and propose contrastive examples. This is a design direction, not an empirical conclusion about pedagogical effectiveness.

For example, a student may believe order never matters in composition. The system offers commutative numerical operations and noncommutative state operations. The question is not whether the student memorizes an exception, but whether they recognize which property of the operations justifies or prohibits changing their order.

The agent would preserve an explicit model of hypotheses about the student's difficulty. A wrong answer may arise from misunderstanding the concept, an arithmetic error or ambiguous wording. It must not automatically become a permanent label of the person's ability.

## Use Case: Interpreting a Condition

A learner confuses “if A, then B” with “if B, then A.” The system generates situations where the original implication is true but its converse false. It then changes the examples' domain to see whether the distinction transfers.

The pedagogical interpreter proposes an explanation and a set of questions. A separate evaluator measures progress on unseen problems. Memory should preserve which explanations were tried and under which conditions they helped, rather than reduce the person to a single score.

MRP's value would lie in selecting pedagogical interventions appropriate to the collision type. But the system must be compared with well-designed teaching methods and an adaptive version without induction of new interpreters. Usefulness cannot be inferred solely from the model's elegance.

## What Must Not Be Optimized

Maximizing correct answers immediately after an explanation may favor memorization. Maximizing time spent in the application may favor dependence on assistance. An educational experiment should track retention, transfer and autonomy under conditions approved for participants.

Human evaluation requires an appropriate protocol and data protection. Before that, the mechanism can be tested with simulated agents having known errors. Those results would validate diagnosis within the model, not demonstrate real pedagogical effects.

The architecture must allow the teacher or learner to challenge the diagnosis. A technical explanation of a “missing distinction” does not justify reducing a complex human difficulty to a single presumed internal representation.

## Agents That Observe and Act

An interactive agent can choose to look, measure, ask or execute an operation. In these situations, interpreters receive not only textual symbols but also images, sensor values or tool results. The MRP model does not exclude these inputs, but must declare which processing procedures are supplied.

A simple example is a simulated device that may be “on,” “faulty” or “pending.” The same indicator light can appear in several states. An image-only interpreter cannot separate the situations. An additional measurement or test command may produce the necessary distinction.

Here, learning an interpreter and choosing an observation become complementary. The new program can specify how to combine the indicator with the test response. But without performing the test, the program obtains no information about the current state. This preserves the separation between computation and data acquisition.

## Use Case: A Workflow Assistant

An assistant can manage steps in an administrative or technical process: request, verification, approval, execution. Initially, it receives explicit states and simulated operations. The goal is to select the next permitted action and detect missing information.

A collision occurs if two requests have identical visible fields but one received an approval that was later revoked. The agent must preserve relevant history or consult the authoritative source. Another collision may arise from the difference between “approved” and “executed.”

A useful system would learn reusable checks for these distinctions without automatically executing actions outside its mandate. Evaluation would measure incorrect steps, unnecessary information requests, abstentions and total cost. Speed is insufficient if the agent skips mandatory conditions.

## Where the Analogy with Understanding Ends

An agent can learn to distinguish a device's states yet fail to understand the social significance of the operation it performs. It may have a representation sufficient for technical tests but insufficient for the consequences for people.

This does not make the model useless. It shows that the family of tests and objectives must be extended carefully and that some judgments remain people's responsibility. Operational understanding always comes with a boundary: for which tasks, observations and consequences is it sufficient?

Interactive applications are promising when we can observe outcomes, limit effects and control the cost of experiments. In opaque or irreversible settings, the same architecture must take a more cautious role, focused on recommendations and clarification.

CHAPTER 23

# The Experimental Program

## The Main Claim and Its Scope

A useful empirical hypothesis must name the tasks, the systems compared and the criterion for success. For the first stage, I would formulate it as follows: within a family of discrete processes with reusable structure, an agent that diagnoses collisions, induces executable distinctions and consolidates them achieves a better error–cost tradeoff than alternatives with comparable access to information and tools.

This claim may be false. The chosen families may not offer enough transfer; induction may be too expensive; the selector may fail; a single model may learn the same structure more efficiently. The protocol must be capable of producing these conclusions too.

We should not begin with the vague objective of "demonstrating understanding." We should begin with observable properties whose combination supports or weakens the proposed interpretation. The name MRP must not become a substitute for measurement.

## Stages and Advancement Thresholds

| Stage | Object of study | Condition for continuing |
|---|---|---|
| 1. Exact collisions | Finite histories, deterministic labels | The diagnosis separates computation from representation loss |
| 2. Controlled induction | Compositions of declared primitives | Refinements resolve unseen structural cases |
| 3. Library | Sequences of related tasks | Reuse amortizes induction and selection |
| 4. Uncertainty | Stochastic observations and incomplete data | Estimates and abstentions are evaluated correctly |
| 5. Controlled texts | Documents generated from known structures | Conditions, attribution and temporality are preserved |

| | | |
|---|---|---|
| 6. Pilot application | Authentic data and domain users | The advantage survives real costs and errors |

Numerical thresholds must be set before the final evaluation, according to the application and a pilot study. I would not invent a mandatory accuracy or a universal savings percentage here. What matters is that the criterion is fixed before the decisive comparison, rather than chosen after the results.

## Baseline Systems

The first baseline is a capable single model trained on the same tasks. The second is a fixed library with a selector. The third is a program generator that solves each task separately, without consolidation. The fourth is the complete MRP system. An ideal baseline that receives the correct representation can be added to estimate how much is lost because of induction.

The ideal baseline is not a realistic competitor and must be labeled accordingly. Its role is diagnostic. If it works but all induction variants fail, the problem is discovering the representation, not the task's impossibility in the chosen language.

When we use an LLM, all relevant variants must receive comparable access to it. Prompts, calls, tokens or other consumption units must be reported, along with local time and memory. It is unfair to count only the final symbolic execution after an expensive external search.

## Ablating the Mechanisms

An ablation removes or replaces a component while keeping the rest as constant as possible. Without diagnosis, the generator receives only the error. Without consolidation, each solution remains local. Without provenance, the agent cannot identify changed dependencies. Without learned selection, we use a simple rule.

If removing a component does not change the results, it does not yet have a demonstrated contribution in the experiment. It may be necessary under other conditions, but that possibility does not justify claiming a current advantage. Ablation should be viewed as a tool for simplification, not a rhetorical obstacle.

For geometry, the alternatives may be an applicability table, a Euclidean representation and a representation with cone-inspired regions. We compare the preservation of relevant relations and the cost, not merely the error of a visual projection.

## Structural Data Splits

Randomly splitting examples can leave the same structure in training and testing. A program then learns a familiar template, and the result is presented as generalization. I would split the data by rules, compositions, dependency lengths and combinations of conditions.

One evaluation could reserve for testing all protocols in which an operation cancels another's effect only for a particular key. Another could reserve combinations of ordering and aggregation. Transfer then measures something stronger than a change of names.

The same principle applies to documents. Fragments from the same document or generated from the same structure must not be treated as entirely independent observations. The sampling unit may be the document, protocol or family, rather than each sentence or each nearly identical trace.

## Statistical Uncertainty

Results must be reported across multiple tasks and initializations, including their variation. An average advantage can hide serious failures in a subfamily. Reporting the distribution of costs and errors, rather than only the mean, is useful.

If a fixed candidate makes no errors in n independent tests from the same distribution, its true error rate has not been proved to be zero. A one-sided 95% confidence bound is 1 − 0.05 to the power 1/n, under the binomial model. The appendix explains the derivation and why it does not automatically apply after adaptive selection on the same tests.

Stopping the experiment precisely when a favorable result appears and omitting earlier attempts can distort the conclusions. The protocol must specify the budget, stopping conditions and model selection method. Research on adaptive validation is relevant to this discipline [Dwork2015].

## What Would Refute the Hypotheses

| Hypothesis | Result that directly weakens it |
|---|---|
| Learned distinctions are general | The advantage disappears on unseen compositions |

| | |
|---|---|
| The library produces savings | The total cost exceeds independent solution |
| Diagnosis is useful | A search without diagnosis achieves the same result |
| Meta-control is worthwhile | A simple rule dominates once selection cost is included |
| Geometry offers useful structure | It does not improve applicability or transfer |
| Translation into circuits helps | Translation errors outweigh the advantage of subsequent verification |

A negative result must be followed by locating its cause, not by arbitrarily changing the definition of success. We may discover that the idea works only for processes with stable rules or for sufficiently reused libraries. A narrower scope, supported by data, is real progress.

## Necessary Artifacts

A research package should contain the task generator, specifications, data distributions, available primitives, induced programs, complete cost logs, evaluator and negative results. For a text study, it would include sources and annotation rules within the limits permitted for their use.

The final report must distinguish demonstrated results, measured results and proposals. This book provides a program and conceptual derivations; it does not present results from such an experimental package. The first technical publication should focus on a narrow, reproducible mechanism before making a claim about intelligence in general.

CHAPTER 24

# Limits, Objections and Contributions

## “Everything Is an Interpreter”

This is the strongest objection of vacuity. If any function that transforms an input is called an interpreter, then any system can be redescribed in MRP vocabulary. Such a general redescription may be philosophically interesting, but it does not discriminate between architectures.

The response is to narrow the empirical program: explicitly represented interpreters, scopes of application, observable collisions, induction mechanisms and transfer criteria. We do not claim that a system lacking these structures cannot understand. We investigate whether these structures produce particular advantages.

The descriptive part can remain a general perspective. The technical part must accept comparison with systems that do not use our terminology. A good result from a monolithic model is relevant information, not a reason to retrospectively redefine the model as MRP to save the hypothesis.

## “It Is Just Operational Semantics”

Operational semantics can describe the system's execution, including learning and changes of interpreters. MRP does not prove that there is a cognitive process impossible to express semantically. Such a claim would require a much stronger argument than the one offered here.

The useful distinction is between describing a system's rules and the problem of selecting or constructing interpretation rules suited to a task. MRP focuses on the latter. It can be formalized semantically without losing its usefulness, just as a learning algorithm can have an exact semantics.

The formulation “semantics are restricted pragmatics” is acceptable as a choice of vocabulary within this framework if it means stabilized regimes of interpretation. It is not a mandatory classification of all theories of meaning and must not be used to downplay the contributions of formal semantics.

## “It Is a Combination of Familiar Ideas”

The objection is largely correct. Behavioral equivalences, abstractions, active learning, program induction and metareasoning have strong precedents. An architecture with multiple modules and a selector is not novel merely because it contains a plurality; hierarchical mixtures of experts offer a clear precedent for dividing inputs among specialized components [Jordan1993].

The difference sought by MRP is not the absence of routing in other systems. It is the traceability of a complete transition: an error is diagnosed as representational insufficiency, a new executable distinction is constructed and checked, and its reuse is then measured. This combination may yield a contribution, but its precise novelty must be demonstrated through comparison and experiment.

| **Existing direction** | **Connection to MRP** | **What should be added or measured** |
|---|---|---|
| Abstract interpretation | Multiple domains for different properties | Incident-guided selection and induction |
| Automata learning | States defined by distinct continuations | Libraries reusable across tasks |
| Predictive representations | State constructed from test predictions | Selection, cost and explicit refinement |
| Program learning | Concepts as executable abstractions | Diagnosis of lost information |
| Mixtures of experts | Specialization and routing | Contracts and the genesis of a new distinction |
| Metareasoning | Choosing computation under costs | The value of building a library |

The table proposes integration questions; it does not claim that no previous system has explored these combinations. The bibliography is selective and does not establish a priority claim. Publishing a new mechanism would require a more focused review of the immediately relevant literature.

## “A Large Model Can Already Do All This”

In principle, a sufficiently expressive model can implement similar procedures internally. MRP does not need monolithic models to be universally incapable in order to be useful. The desired advantage may be control, local verification, cost or adaptation with few examples.

The relevant comparison is operational: what can each system do with the same budget and information? A large model may win. A hybrid system may win. The result may depend on task frequency and error costs. The theory must allow this plurality.

It is also possible that the best technology uses a large model for proposals and small interpreters for repeated execution. This is not a contradiction. It shows that the useful unit of comparison is the complete system and that roles must be attributed correctly.

## “Where Is Understanding Beyond Success?”

Test performance does not exhaust human understanding. An agent may exploit accidental regularities or possess very narrow competence. MRP tries to reduce this problem through contrastive tests, structural transfer and procedure verification, but it does not provide a definitive criterion for consciousness or lived meaning.

The symbol grounding problem remains relevant [Harnad1990]. Linking a representation to observations and interventions provides operational grounding, but does not by definition resolve every philosophical question about meaning. The distinction between an engineering contribution and a complete theory of mind must be preserved.

At the same time, the absence of a complete theory does not prevent testing concrete capabilities. We can prove that a representation loses a distinction and show that a new program recovers it from the available data. This is precise progress, even if it does not answer every question about understanding.

## Limits of Computability and Data

There is no general procedure that decides all interesting semantic properties of arbitrary programs; computability imposes limits, and complexity practically limits many decidable problems [Barak]. Induction and verification must therefore work in restricted languages, with approximations or the possibility of reaching no conclusion.

No more expressive language can create missing empirical information about the current world. No finite library guarantees sufficiency for all future questions. These limits do not require abandoning the program; they require explicit domains and criteria for extension.

A system may be right not to learn a new interpreter. If the task is unique, information is missing or cost exceeds benefit, abstention may be the best option. Constant adaptation is not an end in itself.

## What Is Worth Retaining

The strongest conceptual contribution is the shift in attention from representations viewed as static objects to distinctions constructed through procedures. This formulation's mathematical contribution is modest but clear: we can define sufficiency, collisions, distances and some limits of refinement. The elementary results presented here are not claimed as new theorems in the specialist literature.

The possible engineering contribution is an environment in which these objects are executable, versioned and evaluated. The empirical contribution remains to be established: whether such an environment learns and transfers more efficiently than well-built alternatives.

In its most defensible form, Meta-Rational Pragmatics does not claim to have found the ultimate form of knowledge. It proposes a working question: what relevant distinction can the agent not yet make, what procedure would make it possible, and what evidence shows that it is worth retaining? A real technology can begin right here.

APPENDIX A

# Elementary Mathematical Results

The results below are instructional proofs for the model used in this book. They are not claimed as original theorems. They show exactly where a guarantee arises and which assumptions cannot be removed. We use finite domains when this avoids difficulties with measurability or the existence of an effective representation.

## A1. The Limit of a Representation That Confounds Labels

Consider N examples, each with a correct label, and a representation R. We partition the examples into classes according to their R value. A predictor that sees only R must use the same response distribution for every example in a class.

Within a class, the best deterministic answer is the most frequent label. Randomization cannot achieve a higher average accuracy either: the weighted average of label frequencies does not exceed the maximum frequency. The maximum number of correct answers in the class is therefore the frequency of its most common label.

Minimum empirical error = 1 − (sum of the majority-label frequencies in each class) / N.

For two equally probable histories with the same representation and opposite labels, the minimum error is 0.5. The bound applies to a predictor that receives no other information. If we give it the example's identity or the original history, we have changed the experiment.

## A2. Sufficiency and Response Factorization

Let f be the correct answer to a deterministic question and R a representation on a finite domain. A function g such that f(h) = g(R(h)) for every h exists if and only if all histories with the same R value have the same answer f.

The forward direction is immediate: the same input to g produces the same result. For the reverse direction, we define g at each R value by its class's common answer. The condition guarantees that the definition is unambiguous.

The result asserts the existence of a function, not its efficiency. The table g may be enormous. For infinite domains, the mathematical existence of a factorization does not automatically imply a computable procedure or one accessible to the agent.

## A3. Refinement Does Not Worsen Optimal Risk Without Costs

Suppose the new representation preserves the old one and adds D: Rnew(h) = (R(h), D(h)). Any old predictor can be simulated on the new representation by ignoring component D. Therefore, the lowest possible risk on the new representation cannot be greater than the lowest possible risk on the old one.

The conclusion concerns the optimal predictor within a sufficiently broad class. A concrete algorithm trained on finite data may perform worse after a component is added. Cost, estimation difficulty or overfitting risk may increase. The theorem does not guarantee progress by a real system after every refinement.

## A4. The Number of Strict Refinements in a Finite Sample

A partition of N objects has at most N classes. If we start with K classes and each strict refinement increases the number of classes by at least one, there can be at most N − K strict refinements, provided we do not subsequently merge classes.

The proof is a counting argument. After m refinements we have at least K + m classes; since K + m cannot exceed N, we obtain $m \leq N - K$. The bound says nothing about the cost of finding each refinement. Nor does it say that the final partition generalizes: separating every example may amount only to memorization.

## A5. The Test Pseudometric

For every test t, total variation satisfies TV(P1, P3) ≤ TV(P1, P2) + TV(P2, P3). Each term on the right is at most the corresponding supremum over all tests. Taking the supremum on the left as well, we obtain dT(h1, h3) ≤ dT(h1, h2) + dT(h2, h3).

Nonnegativity and symmetry are inherited directly. The distance can be zero between distinct histories because the tests do not separate them. On classes of histories at zero distance, the function becomes a metric. Adding tests cannot decrease this supremum-defined distance, but may increase the cost of estimating it.

## A6. The 2e Bound for Choosing an Action

Assume a finite set of actions. The estimated value of each action differs from its true value by at most e. Let a* denote a truly optimal action and â an action with the highest estimate. The symbol â is simply a name for the choice based on the estimate.

The true value of a* is at most its estimate plus e. By the definition of the choice, its estimate does not exceed the estimate of â. The estimate of â is at most the true value of â plus e. Combining these three relations, the true value of a* is at most the true value of â plus 2e.

The result requires error control for every action compared. A small average error, without control of the selected action, does not provide the same guarantee. For long plans, what the estimated value of each policy represents must be justified separately.

## A7. Zero Errors Does Not Mean Zero Error Rate

Assume a fixed candidate, independent tests and the same true error probability r on every test. The probability of observing zero errors in n tests is (1 − r) to the power n. We choose the bound r so that this probability is 0.05. This gives r = 1 − 0.05 to the power 1/n.

For n = 100, the bound is approximately 0.0295, or 2.95%. Observing zero errors permits this one-sided 95% confidence bound under the stated model, not the conclusion that error is impossible. For large n, the approximation 3/n is useful because the negative natural logarithm of 0.05 is close to three.

The frequentist interpretation concerns the procedure's coverage over repetitions of the experiment, not a 95% probability assigned to the fixed parameter without a Bayesian model. The bound does not apply unchanged when the candidate was selected after trying many variants on the same tests.

## A8. A Simple Control for a Finite Family of Candidates

We have K candidates fixed before evaluation, where K is a positive integer, and e and d are numbers strictly between zero and one. For any candidate with a true error rate of at least e, the probability of passing n independent tests without error is at most (1 − e) to the power n, which is at most exp(−n × e).

The probability that at least one of the K bad candidates passes is at most the sum of these probabilities: K × exp(−n × e). Independence between candidates is not necessary for this union bound. For the bound to be at most d, it suffices that n be at least [ln(K) + ln(1/d)] / e.

This is a limited guarantee: the class must be fixed independently of the evaluation set, the tests must respect the model, and the criterion is passing without errors. An adaptive search that changes the class without limit in response to results requires another analysis. The formula explains why the number of attempts matters; it does not by itself solve all of MRP validation.

APPENDIX B

# Exercises with Commented Solutions

## B1. Set or Sequence?

Problem: an agent stores only the set of operations “add 1” and “multiply by 2.” Starting from zero, can it determine the result of any two-operation sequence in which each appears once?

Solution: no. Addition followed by multiplication produces two; multiplication followed by addition produces one. The set is the same, but the result differs. A sufficient representation must preserve the order or other information from which the result can be computed. The number of operations does not repair the collision because it is two in both cases.

## B2. What Needs Refining?

Problem: the log preserves the correct order, but the interpreter selects the first event. Is a new representation necessary?

Solution: not for this cause. The necessary information exists. The selection operation must be corrected. A system that adds a new representation may accidentally repair the result, but would miss the diagnosis and introduce unnecessary complexity.

## B3. Noise or Collision?

Problem: the same representation sometimes produces success and sometimes failure. Has its insufficiency been demonstrated?

Solution: no. If the process is stochastic, both outcomes may come from the same distribution. Repetitions or a model showing that two situations with the same representation induce different distributions are necessary. A single disagreement between labels does not distinguish variability from lost information.

## B4. The Missing Relation

Problem: x and y each take values between zero and ten, and the agent must prove that x − y is zero. What additional information is sufficient?

Solution: the relation x = y is sufficient. Narrowing the intervals separately may help in particular cases, but does not generally capture the dependency. The example shows why a relational representation can matter more than greater numerical precision for each variable considered separately.

## B5. When Is Learning Worthwhile?

Problem: constructing and checking an interpreter costs 1,000 ms. The old cost is 40 ms per task, and the new cost, including selection, is 10 ms. How many uses are needed for a strict gain?

Solution: the saving is 30 ms per use. More than 1,000/30 uses are needed, hence 34. At 34, the old cost is 1,360 ms and the new cost 1,340 ms. The result assumes that accuracy remains comparable and that no additional invalidation costs arise.

## B6. A Neighborhood That Is Not an Equivalence

Problem: three binary predictions have probabilities 0.00, 0.06 and 0.12. At threshold 0.08, can we merge all close predictions into a class with diameter at most 0.08?

Solution: no. The first is close to the second and the second to the third, but the first and third are 0.12 apart. The transitive closure of the closeness relation can exceed the threshold. The clustering algorithm must control the diameter or explicitly accept a different guarantee.

## B7. A Procedure's Cone

Problem: two procedures have net utilities (0.8; 0.2) and (0.4; 0.6) in two states. For what probability p of the first state is the first preferable?

Solution: the first has value 0.2 + 0.6p, the second 0.6 − 0.2p. Comparing them gives $0.8p \geq 0.4$, hence $p \geq 0.5$. In unnormalized weights, the condition is $z1 \geq z2$. This defines a cone in the nonnegative quadrant, with equal preference on the boundary.

## B8. Geometric Compression

Problem: we represent a point by $x^2 + y^2$. Can we answer both whether it lies in a concentric annulus and whether it lies to the left of the origin?

Solution: the first question can be answered by comparing the squared radius with the thresholds. The second cannot: points (1; 0) and (−1; 0) have the same representation. The geometry was simplified by losing direction, not by preserving all information.

## B9. Correct Proof, Unsupported Conclusion

Problem: a solver proves that a summary respects the rules extracted from a document. Can we declare the summary faithful to the document?

Solution: only if the correctness of extracting the relevant rules is also justified. Internal verification does not automatically cover translation from text. Two distinct bodies of evidence must be preserved: the link between document and representation, then the link between representation and summary.

## B10. How a Favorite Hypothesis Is Lost

Problem: the MRP system outperforms a simple model but does not outperform a fixed library with a selector once all costs are included. What conclusion is defensible?

Solution: the result supports the usefulness of specialization or selection in the experiment, not the advantage of adaptive induction. We must investigate whether the fixed library already covers all necessary distinctions or whether induction is too costly. It is illegitimate to credit the complete system with an advantage its distinctive component did not produce.

APPENDIX C

# Operational Glossary

**Abstraction.** A representation that preserves some properties and ignores others. Its value depends on the questions for which it is used.

**Abstention.** An outcome in which the agent does not provide the requested answer because information, verification or budget is insufficient.

**Adapter.** A procedure that transforms one representation into another, with a contract concerning the information preserved and lost.

**Agent.** A system that receives observations, maintains a state and chooses operations or actions in relation to tasks and constraints.

**Atlas of interpreters.** A proposed organization of multiple local representations and the adapters between them; it does not assume a geometric manifold.

**Adaptive library.** A versioned collection of interpreters that can be extended, consolidated, suspended and evaluated through reuse.

**Calibration.** Agreement between stated probabilities and the relevant frequencies of outcomes; it is not identical to accuracy.

**Circuit.** An executable composition represented by operations and dependencies between their values.

**Relevant collision.** The identification under the same representation of situations requiring different answers or distributions for the same test.

**Convex cone.** A set closed under linear combinations with nonnegative coefficients; in this book it can describe preference regions among procedures.

**Consolidation.** The promotion and reorganization of procedures for reuse under explicit evaluation and cost criteria.

**Contract.** A specification of an operation's inputs, outputs, preconditions, effects and limits.

**Counterexample.** A case that refutes a precise claim. In CEGAR, whether the abstract example corresponds to concrete behavior must be checked.

**Executable distinction.** A relevant difference that a program can compute from available information and that can be evaluated.

**Behavioral equivalence.** Indistinguishability with respect to a declared family of continuations or tests.

**Episode.** A contextualized record of observations and actions, with the relevant identities and order.

**Execution epoch.** A logical interval during which a task uses stable versions of interpreters and policy.

**Evidence.** An artifact supporting a bounded status: observation, test, derivation, proof or independent evaluation.

**Factorization.** The possibility of obtaining a correct answer through an intermediate representation and a function applied to it.

**Generalization.** Performance on cases not used to construct or select the solution, under explicitly described distributions.

**Program induction.** The construction of candidate programs from examples, specifications and a vocabulary of operations.

**Interpreter.** A procedure that transforms inputs and state into results, new state and evidence under declared conditions.

**Abstract interpretation.** A family of methods for soundly approximating program behavior in domains that preserve selected properties.

**Intervention.** An action that changes a component of the system under study; its effect is generally not identical to observational conditioning.

**Invariant.** A property preserved through a system's relevant transitions under explicit assumptions.

**Meta-controller.** A component that selects computation, observation, induction or stopping operations, taking costs into account.

**MRP.** The research program of constructing, selecting and revising interpreters through verifiable consequences and resource constraints.

**MRP-VM.** A possible execution environment that manages the state, dependencies, interpreters and acceptance conditions of MRP operations.

**Policy.** A rule that chooses actions according to available information; it may include responses to intermediate results.

**Provenance.** Information about a result's sources and transformations; it facilitates auditing but does not guarantee truth.

**Pseudometric.** A distance function that satisfies nonnegativity, symmetry and the triangle inequality but may be zero between distinct objects.

**Refinement.** The addition of distinctions without losing those previously preserved, when the term is used in the strict sense of partitions.

**Predictive representation.** A description of state through predictions about test outcomes, rather than necessarily identifying a latent state.

**SOP Lang.** A language option for executable compositions through circuits, discussed here as possible support for MRP, not as a premise of the theory.

**Sufficiency.** The preservation of information necessary for a declared family of answers or predictions; it does not mean preserving all information.

**Transfer.** The reuse of a capability in other situations or task families, with checks on its conditions of applicability.

**Value of computation.** The expected decision gain produced by an internal operation after relevant costs are subtracted.

**Formal verification.** Establishing a property for all cases covered by a model and the proof's assumptions, not for every possible world.

APPENDIX D

# Annotated Bibliography

The sources below ground the concepts used; they do not validate the MRP architecture as a whole. Identifiers correspond to citations in the text. The original publication year is distinguished from the arXiv submission year where they differ. Addresses were consulted for this edition in September 2026. The selection includes original works, authors' expositions and institutional documentation.

[Angluin1987] Learning Through Queries

Angluin, Dana (1987). Learning regular sets from queries and counterexamples. Information and Computation, 75(2), 87–106. DOI: 10.1016/0890-5401(87)90052-6.

A foundational work on learning regular languages through queries and counterexamples. This book uses it as a precedent for the distinction between a learned model and a test that can distinguish it from the target system, not as a guarantee for every MRP environment.

## Publication record

[Barak] Computability and Complexity

Barak, Boaz. Introduction to Theoretical Computer Science. The author's online textbook, version consulted in September 2026.

A reference for the limits of computability and the difference between a solution's mathematical existence and its efficient computation. These limits motivate restricted languages, budgets and inconclusive outcomes in the proposed architecture.

## The author's textbook

[Boyd2004] Convexity

Boyd, Stephen; Vandenberghe, Lieven (2004). Convex Optimization. Cambridge University Press.

A foundation for convex sets, cones, half-spaces and optimization. The book's derivation of preference regions is an instructional calculation under explicit assumptions about procedures and utilities, not an empirical result about the geometry of knowledge.

## Book page and text made available by the authors

[Callaway2018] Choosing Computations

Callaway, Frederick; Gul, Sayan; Krueger, Paul M.; Griffiths, Thomas L.; Lieder, Falk (2018). Learning to select computations. Proceedings of the Conference on Uncertainty in Artificial Intelligence. Initial preprint: arXiv:1711.06892, 2017.

A direct reference for learning approximate metareasoning policies. It supports positioning the value of computation as an existing foundation, distinct from the MRP hypothesis about constructing and consolidating new interpreters.

## Authors' manuscript

[Clarke2000] Counterexample-Guided Refinement

Clarke, Edmund; Grumberg, Orna; Jha, Somesh; Lu, Yuan; Veith, Helmut (2000). Counterexample-Guided Abstraction Refinement. Computer Aided Verification, Lecture Notes in Computer Science, 1855, 154–169. DOI: 10.1007/10722167_15.

The source for the CEGAR loop. The book distinguishes a spurious abstract counterexample from an ordinary predictive error and proposes an integration inspired by this discipline, without automatically transferring its guarantees.

## Original publication

[Cousot1977] Abstract Interpretation

Cousot, Patrick; Cousot, Radhia (1977). Abstract interpretation: a unified lattice model for static analysis of programs by construction or approximation of fixpoints. Proceedings of the ACM Symposium on Principles of Programming Languages, 238–252.

The foundation of abstract interpretation. In the MRP argument, it shows that a plurality of interpretation domains can have precise soundness obligations and that approximation is not synonymous with an unchecked heuristic.

## Authors' publication page

[CousotIntro] Precision, Approximation and Fixed Points

Cousot, Patrick. Abstract Interpretation in a Nutshell. The author's online introductory exposition.

Complementary reading on the relationship between concrete analysis, abstract domains and stabilization mechanisms. It is recommended before exploring the mathematical details that this book introduces only as needed for the architecture.

## The author's exposition

[DeSabbata2025] Metareasoning for LLMs

De Sabbata, C. Nicolò; Sumers, Theodore R.; AlKhamissi, Badr; Bosselut, Antoine; Griffiths, Thomas L. (2025). Rational Metareasoning for Large Language Models. arXiv:2410.05563, version 3, June 23, 2025; first version submitted in 2024.

A relevant preprint for positioning rational computation selection in systems with language models. The book does not claim that adaptive allocation of reasoning effort alone is an MRP novelty.

## Cited version

[Dwork2015] Adaptive Evaluation

Dwork, Cynthia; Feldman, Vitaly; Hardt, Moritz; Pitassi, Toniann; Reingold, Omer; Roth, Aaron (2015). Generalization in Adaptive Data Analysis and Holdout Reuse. arXiv:1506.02629.

A reference on the risks of adaptively reusing evaluation data. It motivates separating induction, development and final evaluation; it is not used to claim that a single procedure resolves all MRP's statistical risks.

### Authors' manuscript

[Ellis2020] Program Libraries

Ellis, Kevin; Wong, Catherine; Nye, Maxwell; Sable-Meyer, Mathias; Cary, Luc; Morales, Lucas; Hewitt, Luke; Solar-Lezama, Armando; Tenenbaum, Joshua B. (2020). DreamCoder: Growing generalizable, interpretable knowledge with wake-sleep Bayesian program learning. arXiv:2006.08381.

A work connecting the learning of programmatic abstractions to search guidance. It is a concrete precedent for the interpreter library; MRP must separately justify its mechanism for diagnosing, selecting and checking distinctions.

### Manuscript with the cited title

[Ferns2004] Behavioral Distances

Ferns, Norman; Panangaden, Prakash; Precup, Doina (2004). Metrics for Finite Markov Decision Processes. Proceedings of the Conference on Uncertainty in Artificial Intelligence, 162–169. arXiv copy: 1207.4114, submitted in 2012.

A precedent for behavioral metrics in decision processes. The pseudometric defined in this book through the test family is not automatically identified with the exact metric in this work.

### Authors' copy

[Ganea2018] Entailment Cones

Ganea, Octavian; Bécigneul, Gary; Hofmann, Thomas (2018). Hyperbolic Entailment Cones for Learning Hierarchical Embeddings. Proceedings of the 35th International Conference on Machine Learning, PMLR 80, 1646–1655.

A source for representing hierarchical relations through cones. The book separates this motivation, based on partial order, from the derivation of preference cones from expected utilities.

### Publication and authors' materials

[Gardenfors2000] Conceptual Spaces

Gärdenfors, Peter (2000). Conceptual Spaces: The Geometry of Thought. MIT Press. The linked publisher page includes the 2004 paperback edition.

A reference point for representing concepts through quality dimensions and geometric regions. In MRP, a region's shape is treated as a hypothesis dependent on coordinates, operations and tasks.

### Publisher's page

[Gneiting2007] Evaluating Probabilities

Gneiting, Tilmann; Raftery, Adrian E. (2007). Strictly Proper Scoring Rules, Prediction, and Estimation. Journal of the American Statistical Association, 102(477), 359–378. DOI: 10.1198/016214506000001437.

A foundation for scoring probabilistic predictions. The distinction between probability, calibration and utility is necessary to avoid confusing an agent's performance with the honesty or truth of its estimates.

### Text made available by the authors

[Grunwald2004] Description Length

Grünwald, Peter (2004). A tutorial introduction to the minimum description length principle. arXiv:math/0406077.

An introduction to MDL. It supports using description complexity to regularize candidate programs, with the qualification that simplicity depends on the language and does not alone guarantee truth or transfer.

### The author's tutorial

[Harnad1990] Symbol Grounding

Harnad, Stevan (1990). The Symbol Grounding Problem. Physica D: Nonlinear Phenomena, 42, 335–346. DOI: 10.1016/0167-2789(90)90087-6. arXiv copy: cs/9906002, submitted in 1999.

A philosophical reference point for the link between symbols and what they represent. The book proposes operational grounding through sources, observations and interventions, without claiming to solve the entire problem.

### The author's copy

[Jordan1993] Specialization and Routing

Jordan, Michael I.; Jacobs, Robert A. (1993). Hierarchical mixtures of experts and the EM algorithm. Proceedings of the International Joint Conference on Neural Networks, 1339–1344.

A precedent for an architecture with experts and hierarchical weighting mechanisms. The citation refers to the 1993 conference version available at the address below, not the expanded 1994 article with a similar title.

### Conference paper text

[Pearl2018] Observation and Intervention

Pearl, Judea (2018). Theoretical Impediments to Machine Learning With Seven Sparks from the Causal Revolution. arXiv:1801.04016.

A reference for the difference between observational relationships, interventions and causal questions. The book's worked examples are independent and limited; they do not turn predictive representation into a universal causal theory.

### The author's manuscript

[Shannon1948] Information and Coding

Shannon, Claude E. (1948). A Mathematical Theory of Communication. Bell System Technical Journal, 27, 379–423 and 623–656.

A historical foundation for the quantitative treatment of information. The book uses the distinction between the encoded medium and lost information without equating the amount of information with meaning or understanding.

### Academic copy of the original text

[Singh2004] Predictive State Representations

Singh, Satinder; James, Michael R.; Rudary, Matthew R. (2004). Predictive State Representations: A New Theory for Modeling Dynamical Systems. Proceedings of the Conference on Uncertainty in Artificial Intelligence, 512–519. arXiv copy: 1207.4167, submitted in 2012.

A central source for state described through predictions about tests. The book adopts this perspective as a foundation for relevant equivalence, without assuming perfect identification of distributions from finite data.

### Authors' copy

[SoftwareFoundations] Contracts and Correctness

Software Foundations, volume 2, Programming Language Foundations. Chapter Hoare: Hoare Logic, Part I. Collaborative online textbook, version consulted in September 2026.

An instructional reference for preconditions, postconditions and partial correctness. In MRP, a verified contract concerns the model and its assumptions; it does not automatically confirm the truth of external inputs.

### Textbook chapter

[Tappler2019] Learning Reactive Systems

Tappler, Martin; Aichernig, Bernhard K.; Bacci, Giovanni; Eichlseder, Maria; Larsen, Kim G. (2019). L*-Based Learning of Markov Decision Processes (Extended Version). arXiv:1906.12239.

An example of extending active learning to decision processes in the class studied by the authors. The separation between ideal query conditions and their approximation through practical testing is relevant.

### Extended version

[W3C2013] Interoperable Provenance

Moreau, Luc; Missier, Paolo, editors (2013). PROV-DM: The PROV Data Model. W3C Recommendation, April 30, 2013.

A model for describing entities, activities, agents and derivations. It can inform the MRP provenance registry. Recording a derivation enables auditing but does not certify the truth of the derived result.

### W3C Recommendation

[Willsey2021] Rewriting and Equivalence

Willsey, Max; Nandi, Chandrakana; Wang, Yisu Remy; Flatt, Oliver; Tatlock, Zachary; Panchekha, Pavel (2021). egg: Fast and Extensible Equality Saturation. Proceedings of the ACM on Programming Languages, 5, POPL. DOI: 10.1145/3434304. Preprint: arXiv:2004.03082.

A reference for organizing and exploring equivalent expressions through saturation. In an MRP system, using it requires justified rewrite rules; it does not establish arbitrary equivalence between generated programs.

### Authors' manuscript